

Unit 1: Client Side Scripting

- A **scripting language** is a type of programming language which does not require explicit compilation step, and it is designed for a runtime system to automate the execution of tasks.
- For example, a JavaScript program is not needed to be compiled before we run it. These are also known as very high-level programming languages because of working at a high level of abstraction.
- Scripting languages support "script," which is small program written for a specific runtime environment. These are interpreted at runtime rather than compiled. It means, to convert the source code to machine code, scripting languages use an interpreter, not the compiler. As scripting language is not compiled so as we write something meaningful, we can run it immediately.
- The scripting language refers to dynamic high-level, general-purpose interpreted languages such as Python, Perl, etc. Thus, a scripting language can automate different environments such as application softwares, webpages, text editors, operating system shells, computer games, etc.
- JavaScript, Perl, PHP, Python, Ruby, REXX, GameMonkey, etc., are some of the most widely used scripting languages.
- The scripts may be created in two ways: on the *client side* or the *server side*, where the *server-side scripts* are processed on a server. In contrast, *client-side scripting* needs browsers to execute scripts on the client system but doesn't connect with the server executing the client-side scripts.
 - ☑ A **client-side script** is a small program (or set of instructions) that is embedded (or inserted) into a web page. It is processed within the client browser instead of the web server.
 - ☑ Web browsers execute **client-side scripting**. It is used when browsers have all code. Source code is used to transfer from webserver to user's computer over the internet and run directly on browsers. It is also used for validations and functionality for user events.

- ☑ **Client-side scripting** generates code that may be executed on the client end without needing server-side processing. These scripts are typically embedded into HTML text. Client-side scripting may be utilized to check the user's form for problems before submitting it and to change the content based on the user input. The web needs three components to function: client, database, and server.
- ☑ In **Client-side scripting**, source code is visible to the user.

Need of Client Side Scripting Language

- ☞ Client-side scripts are commonly used when we want to validate data before sending it to the web server, adjusting the interface in response to user feedback, and for implementing other advanced features
- ☞ Client-side scripting enables the construction of quicker and more responsive online applications by executing source code on the client's browser rather than the web server.
- ☞ The main advantage of client side scripting is that it is lightweight and relatively easy to implement (syntax not too complex). The editing and executing the code is fast.
- ☞ Data processing is done on the client side from the server, which makes it easier to scale applications with large numbers of users. Thereby, load on the server reduces.
- ☞ The client side data validation can be possible using the client side scripting language like JavaScript.
- ☞ The execution of client side script is more quickly because once the script is downloaded from the server, it is executed by the browser directly on the user's computer.
- ☞ Mathematical assessment is also a possibility by using scripting on the client side.
- ☞ Only script code can be executed by the browser without being connected to the server.
- ☞ Browser will respond instantly when a user presses any key or mouse moves or clicks, etc.

In summary, client-side scripting languages like JavaScript are an essential tool in modern web development, providing the means to create dynamic, interactive, and responsive web applications that provide a better user experience.

Introduction to JavaScript

- JavaScript is the programming language of the web.
- JavaScript is a lightweight, interpreted programming language
- JavaScript was created at Netscape Communications by Brendan Eich in 1995
- JavaScript was initially created to “make web pages alive”.
- When JavaScript was created, it initially had another name: “LiveScript”. But Java was very popular at that time, so it was decided that positioning a new language as a “younger brother” of Java would help.
 - ☞ But as it evolved, JavaScript became a fully independent language with its own specification called **ECMAScript**, and now it has no relation to Java at all.
- In addition to web browsers, databases such as CouchDB and MongoDB uses JavaScript as their scripting and query language.
- JavaScript was traditionally used as client side scripting language. Now it can also be used as server side scripting (Node.js)

Three building blocks in web development: HTML, CSS and JavaScript (JS)

- HTML stands for HyperText Markup Language. "Markup language" means that, rather than using a programming language to perform functions, HTML uses tags to identify different types of content and the purposes they each serve to the webpage.
- CSS stands for Cascading Style Sheets. This dictates how the HTML elements of a website should actually appear on the frontend of the page. HTML provides the raw tools needed to structure content on a website. CSS, on the other hand, helps to style this content so it appears to the user the way it was intended to be seen. These languages are kept separate to ensure websites are built correctly before they're reformatted.
- JavaScript is a programming language that allows you to add interactivity and dynamic behavior to a website
- HTML, CSS, and JavaScript can be compared to the building blocks of a house.
 - **HTML** represents the structure of the house, such as the walls, doors, and windows. It provides the basic layout and content of the website, just like the blueprint of a house.

- **CSS** is like the interior design and decor of the house. It adds style and visual elements to the website, such as colors, fonts, and spacing. CSS enhances the overall look and feel of the website, just like how a well-designed interior can make a house feel more inviting.
- **JavaScript** is like the electrical and plumbing systems of the house. It provides interactivity and dynamic functionality to the website, such as form validation, drop-down menus, and animations. Just as an electrical system powers the lights and appliances in a house, JavaScript powers the interactive elements of a website.
- Together, **HTML, CSS, and JavaScript form a complete website**, just like how **walls, interior design, and electrical systems make a complete house**

Features of JavaScript

- All popular web browsers support JavaScript as they provide built-in execution environments.
- JavaScript follows the syntax and structure of the C programming language. Thus, it is a structured programming language.
- JavaScript is a weakly typed language, where certain types are implicitly cast (depending on the operation).
- JavaScript is an object-oriented programming language that uses prototypes rather than using classes for inheritance.
- It is a light-weighted and interpreted language.
- It is a case-sensitive language.
- JavaScript is supportable in several operating systems including, Windows, macOS, etc. It provides good control to the users over the web browsers.

Applications of JavaScript programming

- Javascript is one of the most widely used programming languages (Front-end as well as Back-end). It has its presence in almost every area of software development. Some of them are listed here:

- **Client side validation** - This is really important to verify any user input before submitting it to the server and Javascript plays an important role in validating those inputs at front-end itself.
- **Manipulating HTML Pages** - Javascript helps in manipulating HTML page on the fly. This helps in adding and deleting any HTML tag very easily using javascript and modify your HTML to change its look and feel based on different devices and requirements.
- **User Notifications** - You can use Javascript to raise dynamic pop-ups on the webpages to give different types of notifications to your website visitors.
- **Back-end Data Loading** - Javascript provides Ajax library which helps in loading back-end data while you are doing some other processing. This really gives an amazing experience to your website visitors.
- **Presentations** - JavaScript also provides the facility of creating presentations which gives website look and feel. JavaScript provides RevealJS and BespokeJS libraries to build a web-based slide presentations.
- **Server Applications** - Node JS is built on Chrome's Javascript runtime for building fast and scalable network applications. This is an event based library which helps in developing very sophisticated server applications including Web Servers.

Advantages of JavaScript

- ⇒ **Less server interaction** – You can validate user input before sending the page off to the server. This saves server traffic, which means less load on your server.
- ⇒ **Immediate feedback to the visitors** – They don't have to wait for a page reload to see if they have forgotten to enter something.
- ⇒ **Increased interactivity** – You can create interfaces that react when the user hovers over them with a mouse or activates them via the keyboard.
- ⇒ **Richer interfaces** – You can use JavaScript to include such items as drag-and-drop components and sliders to give a Rich Interface to your site visitors.

Formatting and Coding Conventions in JavaScript

- In JavaScript, formatting and coding conventions are a set of guidelines that developers follow to write code that is easy to read, maintain, and understand.
- These conventions help to ensure that code written by different developers is consistent and follows best practices.
- Some of the most important formatting and coding conventions in JavaScript include:
 - ☑ **Indentation:** JavaScript code blocks should be indented using either tabs or spaces, with a common convention being to use 2 or 4 spaces for each level of indentation.
 - ☑ **Line Length:** It is recommended to keep lines of JavaScript code to a maximum of 80 characters in length. When a line of code exceeds this length, it can be broken into multiple lines using the backslash () character.
 - ☑ **Semicolons:** JavaScript statements should be terminated with a semicolon (;). Although *semicolons are optional in JavaScript*, they are widely used to make code easier to read and maintain.
 - ☑ **Naming Conventions:** Variables and functions in JavaScript should be named using camelCase or PascalCase conventions, while constants should be written in ALL_CAPS.
 - ☑ **Comments:** JavaScript code should include comments that explain what the code is doing. Comments can start with two forward slashes (//) and continue to the end of the line, or they can be written as block comments, which start with /* and end with */.
 - ☑ **Consistency:** It is important to maintain a consistent style throughout the codebase, including consistent use of indentation, naming conventions, and commenting practices.
- By following these and other formatting and coding conventions, developers can write JavaScript code that is easier to read, maintain, and understand, and that can be used and maintained by a wider community of developers. Additionally, using consistent conventions helps to ensure that code is maintainable and scalable, even as it grows in size and complexity.

JavaScript Files

- In JavaScript, files are used to store and organize code, making it easier to maintain and reuse code across multiple projects. A JavaScript file can contain any valid JavaScript code, including variables, functions, objects, arrays, and other data types.
- There are two main types of JavaScript files:
 - **.js file:** A .js file (also known as a JavaScript file) is a plain text file that contains JavaScript code. The code in a .js file can be included in an HTML file using the `<script>` tag, which specifies the source of the JavaScript code.
 - **.json file:** A .json file (also known as a JSON file) is a plain text file that contains data in JavaScript Object Notation (JSON) format. JSON is a data interchange format that is often used to store and exchange data between a server and a client in a web application.
- In general, a JavaScript file is used to contain logic and functionality that is executed by the client-side, while a JSON file is used to store and exchange data between the client and server. By organizing code into separate files, it is easier to maintain and reuse code, and to separate the logic of an application from its data.

Embedding JavaScript in HTML

- ❖ JavaScript can be embedded in HTML using the `<script>` tag. The `<script>` tag is used to specify the source of the JavaScript code that should be executed in the web page. There are two main ways to include JavaScript in an HTML page:
 - a) **Internal Scripts:** Internal scripts are placed within the HTML document itself and are defined using the `<script>` tag. The code is placed between the opening `<script>` and closing `</script>` tags. For example:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Example Page</title>
  </head>
  <body>
    <h1>Welcome to my website!</h1>
    <script>
      alert("Hello World!");
    </script>
  </body>
</html>
```

- b) **External Scripts:** External scripts are stored in a separate .js file and are included in the HTML document using the <script> tag and the "src" attribute. The "src" attribute specifies the location of the external .js file. For example:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Example Page</title>
  </head>
  <body>
    <h1>Welcome to my website!</h1>
    <script src="script.js"></script>
  </body>
</html>
```

Using Script Tag

- ❖ The <script> tag is used in HTML to embed and execute JavaScript code within a web page. The basic syntax for the <script> tag is as follows:

```
<script>
  JavaScript code goes here
</script>
```

- ❖ You can place the <script> tag anywhere within the HTML document, but it is generally recommended to include it just before the closing </body> tag. This ensures that the JavaScript code will be executed after the rest of the page has loaded, preventing any potential delays or issues with the page's content.

- ❖ We can also include external JavaScript files in your HTML document using the **<script>** tag. To do this, you would use the src attribute to specify the location of the external JavaScript file:

```
<script src="file.js"></script>
```

- ❖ This will include the code from the "file.js" file and execute it as if it were directly included within the HTML document. This makes it easy to reuse code across multiple pages and can help to keep your code organized and maintainable.
- ❖ The script tag takes two important attributes –
 - **Language** – This attribute specifies what scripting language you are using. Typically, its value will be javascript. Although recent versions of HTML (and XHTML, its successor) have phased out the use of this attribute.
 - **Type** – This attribute is what is now recommended to indicate the scripting language in use and its value should be set to "text/javascript".
 - So your JavaScript segment will look like –

```
<script language = "javascript" type =  
"text/javascript">
```

JavaScript code

```
</script>
```

NoScript Tag

- ❖ The **<noscript>** tag is used in HTML to provide an alternate content for browsers that do not support JavaScript or have JavaScript disabled. The **<noscript>** tag should contain content that will be displayed to the user if JavaScript is not available.
- ❖ The basic syntax for the **<noscript>** tag is as follows:

```
<noscript>
  Alternate content goes here
</noscript>
```

- ❖ For example, you might use the **<noscript>** tag to display a message to the user indicating that JavaScript is required to view the page or to provide a limited version of the page that does not require JavaScript.
- ❖ Here is an example of how you might use the **<noscript>** tag in a web page:

```
<script>
  // JavaScript code here
</script>
<noscript>
  <p>JavaScript is required to view this page. Please enable
  JavaScript in your browser and try again.</p>
</noscript>
```

- ❖ In this example, the JavaScript code between the **<script>** tags will be executed if the user's browser supports JavaScript and has it enabled. If JavaScript is not available, the content between the **<noscript>** tags will be displayed instead.

JavaScript Runtime Environments

- A **JavaScript runtime environment** is the environment in which JavaScript code is executed. It provides the necessary infrastructure for executing JavaScript code and includes things like the JavaScript engine, built-in libraries, and APIs for accessing the underlying platform. The runtime environment is responsible for executing the code, managing memory and other resources, and providing access to the underlying platform through APIs.
- A **JavaScript engine** is the part of a web browser or standalone JavaScript interpreter that executes JavaScript code. It is responsible for parsing the JavaScript code and executing the commands specified in the code. The **most widely used JavaScript engines** are **V8** (used by Google Chrome), **Spider Monkey** (used by Mozilla Firefox), and **Chakra** (used

by Microsoft Edge). These engines are designed to execute JavaScript code quickly and efficiently.

- **The most common JavaScript runtime environment is the web browser**, which provides a variety of APIs for accessing the Document Object Model (DOM), making HTTP requests, working with local storage, and more. In this environment, the JavaScript engine is integrated into the browser and executes JavaScript code that is included in web pages.
- **Another common JavaScript runtime environment is a standalone JavaScript interpreter, such as Node.js**, which provides a server-side JavaScript environment. In this environment, the JavaScript engine is not integrated into a web browser but instead runs as a standalone process. Node.js provides a variety of APIs for working with the file system, making HTTP requests, and more, in addition to the JavaScript language itself.

Here are a few examples of JavaScript runtime environments:

- **Web Browsers:** Most web browsers, such as Google Chrome, Mozilla Firefox, and Microsoft Edge, provide a JavaScript runtime environment as part of their built-in JavaScript engine. These runtime environments provide the necessary infrastructure for executing JavaScript code and also provide APIs for accessing and manipulating the Document Object Model (DOM) of a web page.
- **Node.js:** Node.js is a standalone JavaScript runtime environment that is designed for server-side programming. It provides a JavaScript runtime environment that is built on Chrome's V8 JavaScript engine, but extends it with additional server-side APIs for accessing the file system, making network requests, and working with databases.
- **Deno:** Deno is a relatively new runtime environment for JavaScript that aims to provide a secure and modern alternative to Node.js. It is built on the V8 JavaScript engine and provides a built-in module system, a standard library, and access to the underlying platform through APIs.
- **J2V8:** J2V8 is a JavaScript runtime environment for the Java Virtual Machine (JVM). It provides a way to execute JavaScript code within a Java application,

allowing developers to take advantage of the capabilities of both JavaScript and Java

JavaScript libraries

- JavaScript libraries are pre-written collections of JavaScript code that provide a set of functions and features that make it easier to perform common tasks, such as animation, handling user events, making HTTP requests, and more. By using a library, developers can save time and effort, as they don't need to write all of the code themselves.
- Some popular JavaScript libraries include:
 - ☑ **jQuery:** A widely used library that makes it easier to work with the Document Object Model (DOM), handle user events, make animations, and perform AJAX requests.
 - ☑ **React:** A library for building user interfaces that is maintained by Facebook. React makes it easy to build complex UIs by breaking them down into small, reusable components.
 - ☑ **Angular:** A complete JavaScript framework for building web applications. Angular provides a large set of features for building dynamic UIs, including two-way data binding, a powerful templating system, and a rich set of built-in directives.
 - ☑ **Vue.js:** A progressive JavaScript framework for building user interfaces. Vue.js is designed to be simple and flexible, and is often used for building single-page applications and other dynamic UIs.

Operators in JS

- ☞ An operator in JavaScript is a symbol that performs a specific operation on one or more operands (values/variables)
- ☞ There are following types of operators in JavaScript.
 - ☑ **Arithmetic Operators**
 - ☑ **Comparison (Relational) Operators**
 - ☑ **Bitwise Operators**
 - ☑ **Logical Operators**
 - ☑ **Assignment Operators**

☑ Special Operators

Arithmetic Operators

- ☑ Arithmetic operators are used to perform arithmetic operations on the operands. Here is a list of operators that are known as JavaScript arithmetic operators:

Operator	Description	Example
+	Adds two operands	10 + 20 = 30
-	Subtracts the second operand from the first	30 - 20 = 10
/	Divide the numerator by the denominator	20/10 = 2
*	Multiply two operands	5 * 5 = 25
%	Outputs the remainder of an integer division	20 % 10 = 0
++	Increases an integer value by one	let a=20; a++; Now a = 21
--	Decreases an integer value by one	let a=20; a- -; Now a = 19

Comparison Operators

- ☞ The JavaScript comparison operator compares the two operands. The comparison operators are as follows:

Operator	Description	Example
==	Checks if two operands are equal or not. If yes, then the condition becomes true.	20 == 30 -> false
===	Finds the identical (equal and of the same type)	10 === '10' -> false
!=	Checks if two operands are equal or not. If the values are not equal, then the condition becomes true	20 != 30 » true
!==	It implies that two values are not Identical	20 !== 20 -> false

>	Checks if the value of the left operand is greater than the value of the right operand	30>10 »true
> =	Checks if the value of the left operand is greater than or equal to the value of the right operand	20>=10 ->true
<	This Checks if the value of the left operand is less than the value of the right operand	20<10 -> false
< =	Checks if the value of the left operand is less than or equal to the value of the right operand	30<=10 ->>false

Bitwise Operators

The bitwise operators are used to perform bitwise operations on operands. Here is a list of bitwise operators:

Operator	Description	Example
&	Boolean AND operation on each bit of its integer arguments	(10==20 & 20==33) -> false
	It performs a Boolean OR operation on each bit of its integer arguments	(10==20 20==33) ->false
^	This operator performs Bitwise XOR operation	(10==20 ^ 20==33) -> false
~	It is a unary operator and operates by reversing all the bits in the operand	(~10) = -11
<<	Moves all the bits in its first operand to the left by the number of places specified in the second operand.	(10<<2) = 40
>>	The left operand's value is moved right by the number of bits specified by the right operand.	(10>>2) = 2
>>>	This operator is just like the >> operator, except that the bits shifted in on the left are always zero.	(10>>>2) = 2

Logical Operators

- The logical operators are used to combine two or more conditions. JavaScript provides the following logical operators:

Operator	Description	Example
&&	Logical AND – If both the operands are non-zero, then the condition becomes true	(10==20 && 20==33) -> false
 	Logical OR – If any of the two operands are non-zero, then the condition becomes true.	(10==20 20==33) ->false
!	Logical NOT – Reverses the logical state of its operand.	!(10==20) ->true

Assignment Operators

- The Assignment operators are used to assign values to the operand. The following operators are known as JavaScript assignment operators:

Operator	Description	Example
=	Assigns values from the right side operand to the left side operand	20+10 = 30
+=	It adds the right operand to the left operand and assigns the result to the left operand	let a=20; a+=10; Now a = 30
-=	It subtracts the right operand from the left operand and assigns the result to the left operand	let a=30; a-=10; Now a = 20
=	It multiplies the right operand with the left operand and assigns the result to the left operand	let a=10; a=20; Now a = 200
/=	It divides the left operand with the right operand and assigns the result to the left operand	let a=10; a/=2; Now a = 5
%=	It takes modulus using two operands and assigns the result to the left operand	let a=10; a%=2; Now a = 0

Other JavaScript Operators

- Here's a list of some other operators available in JavaScript.

Operator	Description	Example
,	evaluates multiple operands and returns the value of the last operand.	let a = (1, 3, 4); // 4
?:	returns value based on the condition	(5 > 3) ? 'success' : 'error'; // "success"
delete	deletes an object's property, or an element of an array	delete x
typeof	returns a string indicating the data type	typeof 3; // "number"
void	discards the expression's return value	void(x)
in	returns true if the specified property is in the object	prop in object
instanceof	returns true if the specified object is of the specified object type	object instance of object_type

Variables in JS

- ☒ The term “variable” implies that new values can be assigned: that the value associated with the variable may vary as our program runs.

- ☞ If we permanently assign a value to a name, then we call that name a **constant** instead of a variable.
- ☑ In JavaScript, a variable is a container for storing data values. Variables can store different types of data, such as numbers, strings, arrays, and objects, among others.
- ☑ Before you can use a variable or constant in a JavaScript program, you must declare it. In ES6 and later, this is done with the **let** and **const** keywords, which we explain next.
 - ☞ Prior to ES6, variables were declared with **var**.
- ☑ Here are the important rules that must be followed while declaring a variable in JavaScript.
 - ☞ In JavaScript variable names are case sensitive i.e. a is different from A.
 - ☞ Variable name can only be started with a underscore (_) or a letter (from a to z or A to Z), or dollar (\$) sign.
 - ☞ Numbers (0 to 9) can only be used after a letter.
 - ☞ No other special character is allowed in variable name.

let vs var vs const

- ☑ In JavaScript, var, let, and const are keywords used to declare variables. While they all serve the same purpose of storing values, there are some key differences between the three that affect how and where the variables can be used.
 - **var:** The var keyword is used to declare variables in JavaScript and has function scope. This means that variables declared with var are accessible within the entire function, and can be re-declared within the same function. The var keyword is often considered outdated, as it can lead to issues with variable hoisting and confusing scoping rules.

```
function example() {  
  var x = 10;  
  if (true) {  
    var x = 20;  
    console.log(x); // outputs 20  
  }  
  console.log(x); // also outputs 20  
}
```

- **let:** The let keyword was introduced in ECMAScript 6 (ES6) as an alternative to **var**. It allows you to declare variables with block-level scope, which means that a variable declared with *let* is *only accessible within the block in which it was declared*. This makes it easier to manage the scope of variables and helps avoid unintended side effects.

```
function example() {  
  let x = 10;  
  if (true) {  
    let x = 20;  
    console.log(x); // outputs 20  
  }  
  console.log(x); // outputs 10  
}
```

- **const:** The const keyword is also introduced in ES6, and it allows you to declare variables that *cannot be re-assigned*. In other words, a constant value is set once, and cannot be changed. const also has block-level scope, meaning that it is only accessible within the block in which it was declared.

```
const PI = 3.14;
PI = 3.15; // throws an error, as you cannot re-assign a constant value
```

- ☑ In general, *it's recommended to use const whenever you want to declare a variable that won't change*, and *use let for variables that will be updated or re-assigned within the block*. If you need *to support older browsers or environments, you can use var*, but be aware of its limitations and potential side effects.

	var	let	const
can be redeclared	yes	no	no
can be reassigned	yes	yes	no
can be read outside of its own scope	yes	no	no

Variable Hoisting

- In JavaScript, variable hoisting is a mechanism where variables and function declarations are moved to the top of their respective scopes before the code is executed.
- This means that you can use a variable before it has been declared, and it will still work as expected, because the variable declaration has been "hoisted" to the top of the scope. For example:

```
x = 5;

console.log(x); // Output: 5

var x;
```

- ☞ In the above code, the variable `x` is assigned the value 5 before it is declared using the `var` keyword. However, this code still works because the declaration of `x` is hoisted to the top of the scope before the code is executed.
- It's important to note that only the declaration of the variable is hoisted, not its assignment. So if you try to access a variable before it has been declared and assigned a value, you will get an error. For example:

```
console.log(x); // Output: ReferenceError: x is not defined  
  
var x = 5;
```

- In general, it's best practice to declare all your variables at the top of their respective scopes to avoid any confusion or unexpected behavior.

JavaScript Data Types

- JavaScript provides different data types to hold different types of values.
- JavaScript types can be divided into two categories: **primitive types** and **non-primitive (Complex/reference) types**.
- Variables in JavaScript are **dynamically typed**, which means they can hold any data type, and their type can change at runtime. For example, a variable that initially holds a number can be reassigned to hold a string later on.

Some Primitive data types:

- **string**: represents a sequence of characters
- **number**: represents a numeric value
- **boolean**: represents a logical value of true or false
- **null**: represents an intentional absence of any object value
- **undefined**: represents a declared variable without a value or an undefined value

Some Non-primitive/Complex/ reference data types

- **object:** represents a collection of key-value pairs, where the keys are strings (or Symbols) and the values can be any data type, including other objects
- **function:** represents a special type of object that can be called and executed

JavaScript Output

☑ JavaScript can "display" data in different ways like:

- *Writing into an HTML element, using `innerHTML`.*
- *Writing into the HTML output using `document.write()`.*
- *Writing into an alert dialog box, using `window.alert()`.*
- *Writing into an prompt dialog box using `window.prompt()`*
- *Writing into an confirm dialog window `confirm()`*
- *Writing into the browser console, using `console.log()`*

Using innerHTML

- To access an HTML element, JavaScript can use the **`document.getElementById(id)`** method.
- The **`id`** attribute defines the HTML element. The **`innerHTML`** property defines the HTML content:
- We can write output to a specific element on the page by using the `innerHTML` property. This property sets the HTML content within an element.

```
<!DOCTYPE html>
<html>
<body>
    <h1>My First Web Page</h1>
```

```

<p> I am Studying Scripting Language (SL) </p>
<p id="demo"></p>
<script>
    document.getElementById("demo").innerHTML = 4 + 3;
</script>
</body>
</html>

```

Using document.write()

- For testing purposes, it is convenient to use ***document.write()***
- This method writes output directly to the HTML document. However, it is not recommended to use this method in modern web development, as it can overwrite the entire page and cause issues with scripts that manipulate the DOM after the page has loaded.

```

<!DOCTYPE html>

<html>

<body>

    <h2>My First Web Page</h2>

    <p>My first paragraph.</p>

    <p>Never call document.write after the document has
    finished loading.It will overwrite the whole document.

    </p>

    <script>

        document.write(15 - 8);

    </script>

```

```
</body>
```

```
</html>
```

Using `window.alert()`

- The **`alert()`** method displays a pop-up window with the specified message. It is often used for debugging purposes or to display simple messages to the user.

```
alert("Hello, World!");
```

NOTE: We can skip the `window` keyword. In JavaScript, the `window` object is the global scope object. This means that variables, properties, and methods by default belong to the `window` object. This also means that specifying the `window` keyword is optional.

Using `window.prompt()`

The **`window.prompt()`** method displays a dialog box that prompts the user to enter a value. This method can be useful for gathering user input.

```
const name = prompt("Please enter your name:");
```

Using `window.confirm()`

- The **`confirm()`** method displays a dialog box with a message and two buttons (OK and Cancel). This method can be used to ask the user to confirm an action, and it returns a boolean value indicating whether the user clicked OK or Cancel.

```
const result = confirm("Are you sure you want to delete this item?");
```

Using console.log()

- The most common method for outputting data in JavaScript is by using the console.log() method. This method writes the output to the JavaScript console, which can be viewed in a web browser's developer tools.

```
console.log("Hello, World!");
```

Control Structures in JS

- ⇒ JavaScript programs are nothing more than a sequence of statements to execute.
- ⇒ By default, the JavaScript interpreter executes these statements one after another in the order they are written. Another way to “make something happen” is to alter this default order of execution, and JavaScript has a number of statements or **control structures** that do just this.
- ⇒ Control structure controls the flow of execution of a program.
- ⇒ JavaScript has number of statements that control flow of the program. They are :

a) Conditionals

- Statements like **if** and **switch** that make the JavaScript interpreter execute or skip other statements depending on the value of an expression
- **if , if- else, if – else- if , switch-case**

b) Loops

- Statements like **while** and **for** that execute other statements repetitively
- **while , do –while, for, for-of , for-in**

c) Jumps

- Statements like **break**, **return**, and **throw** that cause the interpreter to jump to another part of the program
- **break , continue, return, throw, try, catch finally, yield**

JavaScript if Statement:

- If statement is used to execute a block of statements if specified condition is true.
- If the condition is true, the code is executed. If not, then the control moves on to the following code.
- Syntax:

```
if(condition){  
  
    //Block of JavaScript statements.  
  
}
```

JavaScript if-else Statement

- if-else statement is used to execute either of two block of statements depends upon the condition. If condition is true then if block will execute otherwise else block will execute.
- The first statement is executed when the condition is true; if not, the else statement is executed. Then the control moves on to the following code.
- Syntax:

```
if(condition){  
  
    //Block of JavaScript statements1.  
  
}else{  
  
    //Block of JavaScript statements2.  
  
}
```

JavaScript if -else -if Statement

- **else-if** statement is used to execute one block of statements from many depends upon the condition. If condition1 is true then block of statements1 will be executed, else if condition2

is true block of statements2 is executed and so on. If no condition is true, then else block of statements will be executed.

- In contrast to **if-else**, we can include a variety of conditions in **else-if**. The control checks the expression from top to bottom, goes to the next if one is false, and eventually finishes with the **else** statement, which acts as a default statement when none of the conditions are met.
- **Syntax:**

```
if(condition1){  
  
    //Block of JavaScript statements1.  
  
}else if(condition2){  
  
    //Block of JavaScript statements2.  
  
} . . . else if(conditionn){  
  
    //Block of JavaScript statementsn.  
  
}else{  
  
    //Block of JavaScript statements.  
  
}
```

Example

```

<html>
  <head>
    <title>MyPage</title>
  </head>
  <body>
    <h4>To Demonstrate JavaScript Conditional Statements</h4>
    <h6 id="line1"></h6>
    <h6 id="line2"></h6>
    <h6 id="line3"></h6>
  <script >
    let num = 60;
    //if statement
    if(num>50){
      document.getElementById("line1").innerHTML = num + " is greater than 50";
    }

    //if -else
    if(num%2 == 0){
      document.getElementById("line2").innerHTML = num + " is EVEN NUMBER ";
    }else{
      document.getElementById("line2").innerHTML = num + " is ODD NUMBER ";
    }

    //if-else-if
    if(num < 0 ){
      document.getElementById("line3").innerHTML = num +"is NEGATIVE NUMBER";
    }else if(num < 10){
      document.getElementById("line3").innerHTML=num +"is SINGE DIGIT
NUMBER";
    }else if(num < 100){
      document.getElementById("line3").innerHTML= num + " is DOUBLE DIGIT
NUMBER";
    }else {
      document.getElementById("line3").innerHTML = num + " is MULTI DIGIT
NUMBER";

```

```
    }  
  </script>  
</body>  
</html>
```

Switch Statement

- The switch statement is similar to the if-else statement. If all of the branches depend on the value of the same expression, the switch control structure can be used instead of the if-else

```
switch (expression) {  
  
    case value1:  
  
        //Block of JavaScript statements1.  
  
        [break;]  
  
    case value2:  
  
        //Block of JavaScript statements2.  
  
        [break;]  
  
    case valuen:  
  
        //Block of JavaScript statementsn  
  
        [break;]  
  
    default:  
  
        //Block of JavaScript defaultstatements.  
  
}
```

- When the switch statement is executed, the program attempts to match the value of expression to a case value. If the value of expression is equal to a case value, the program executes the associated statements. If matching is not found, the program looks for the optional default clause. If it is found, associated statements are executed, if not the program skips all statements within the switch statement.

- The optional break keyword associated with each case ensures that the program breaks out of the switch statement once the matched statement is executed. If the break is omitted all subsequent statements will also be executed.
- The break statement after every case statement is to let the control know the end of the statement. If the break is not added, the control will end up executing every statement.

```
<h5>JS switch-case Example</h5>
<h6 id = "display"> </h6>
<script>

    let a = 1;
    let res = 0;

    switch (a) {
        case "1":
            res= 1;
            break;
        case 1:
            res= 'one';
            break;
        case 2:
            res = 'two';
            break;

        default:
            res = 'not found';
            break;
    }
    document.getElementById("display").innerHTML="The value is "+res;
</script>
```

Task: Take operators from prompt and display results by performing corresponding operations.
(Simple Calculator Program)

for loop in JS

- The JavaScript for loop iterates the elements for the fixed number of times. It should be used if number of iteration is known.
- A for loop in JavaScript is a way to iterate over a set of values for a specific number of times. Here is the basic syntax for a for loop in JavaScript:

```
for (initialization; condition; update) {

    // code to be executed

}
```

- The *initialization* is where you declare and set the initial value of the variable that will be used to keep track of the loop iteration.
- The *condition* is evaluated at the beginning of each iteration. If it is true, the code within the loop is executed. If it is false, the loop stops.
- The *update* is executed at the end of each iteration and is used to update the variable that keeps track of the loop iteration.

Example:

```
<h2>JavaScript For Loop</h2>

<p id="demo"></p>
<script>
    let text = "";

    for (let i = 0; i < 5; i++) {
        text += "The number is " + i + "<br>";
    }

    document.getElementById("demo").innerHTML = text;

</script>
```

JS while loop

- A while loop in JavaScript is another way to repeatedly execute a block of code while a condition is true.
- A while statement executes its statements as long as a specified condition evaluates to true
- Here's the basic syntax for a while loop in JavaScript:

```
while (condition) {  
  
    // code to be executed  
  
}
```

- The **condition is checked at the beginning of each iteration**. If it is true, the code within the loop is executed. If it is false, the loop stops.
- A **while loop** is useful when you don't know the exact number of times a loop should run, but you do know the condition that should cause it to stop. If you know the exact number of times a loop should run, then a for loop is typically a better choice.

<h2>JavaScript While Loop</h2>

<p id="demo"></p>

```
<script>  
    let text = "";  
    let i = 0;  
    while (i < 10) {  
        text += "<br>The number is " + i;  
        i++;  
    }  
    document.getElementById("demo").innerHTML = text;  
</script>
```

JS do- while loop

- A do-while loop in JavaScript is similar to a while loop, except that the code within the loop is executed at least once, even if the condition is initially false. Here's the basic syntax for a do-while loop in JavaScript:

```

do {

    // code to be executed

} while (condition);

```

- The code within the loop is executed first, and then the condition is checked. If the condition is true, the loop will execute again. If the condition is false, the loop stops.
- The key difference between a do-while loop and a while loop is that a **do-while loop guarantees that the code within the loop will be executed at least once**, regardless of the condition.

Note: **do-while** loop is similar to the **while loop**. The only difference is that in **do-while** loop, the body of loop is executed **at least once**.

```

<h2>JavaScript Do While Loop</h2>
<p id="demo"></p>
<script>
let text = ""
let i = 0;
do {
    text += "<br>The number is " + i;
    i++;
}
while (i < 10);
document.getElementById("demo").innerHTML = text;

```

</script>

Array in JavaScript

- ☑ In JavaScript, an array is a special type of object that allows you to store and manipulate a collection of values, which can be of any data type.
- ☑ An array is an ordered collection of data. It is a container that holds multiple values. Each element of the array has a fixed position called an index and stores a value. All indexes are numeric, the first element has index 0, second 1, etc.
- ☑ Arrays in JavaScript are dynamic and increase or decrease as needed. Their size does not have to be predetermined.
- ☑ JavaScript arrays may be sparse – there can be gaps in the indexes.
- ☑ Another feature of arrays in JavaScript is that they are untyped, elements of an array can be of different types.
- ☑ In fact JavaScript arrays are a specialized form of objects. So indexes are property names that are integers.
- ☑ JavaScript provides a predefined **Array** object that you can use to work with arrays.

Creating arrays

⇒ There are several ways to create arrays. We can **create arrays** with:

- **Array literals**
- **The ... spread operator on an iterable object**
- **The Array() constructor**
- **The Array.of() and Array.from() factory methods**

Creating arrays with Array literals

- ✎ **Array literal** is a list of array elements separated by commas and enclosed within square brackets
- ✎ To create an **empty array** simply use empty square brackets:


```
let emptyArray = [];
```
- ✎ To create an array of elements enclose them in **square brackets**:

```
const numbers = [1,3, 40, 102];  
const texts = ["IT", "software", "engineer", "dev"];
```

- ✎ In one array you can enclose **values of different types**:

```
let mixture = [0, 10.5, true, "child"];
```

- ✎ An array **can contain objects and other arrays**:

```
let myArray = [{age: 20, name: 'Mahendra'}, ["good", "strong"]];
```

Creating arrays with Array () Constructor

- ✎ Another way of creating arrays is to use the Array constructor

- ✎ Creating an empty array:

```
let emptyArray = new Array();
```

- ✎ To create an array of elements pass more than one numeric value or non-numeric value as constructor arguments:

```
let array1 = new Array ('a'); // single element array
```

```
let array2 = new Array(4, 8, 12); // array of numbers
```

```
let array3 = new Array(1, 3.14, false, "bca"); // array of  
different types
```

- ✎ If you pass a single numeric value as an argument, it is treated as an array length specification.

```
let emptyArray = new Array(5);
```

This creates an empty array with the specified length.

Note that calling the constructor `Array` with a single numeric argument, which is not an integer, causes an error.

```
let myArray = new Array(3.5); // RangeError: invalid array length
```

Creating arrays using `Array.of()` Method

- ✎ When the **`Array()`** constructor function is invoked with one numeric argument, it uses that argument as an array length. But when invoked with more than one numeric argument, it treats those arguments as elements for the array to be created.
 - This means that the `Array()` constructor cannot be used to create an array with a single numeric element.
- ✎ In ES6, the **`Array.of()`** function addresses this problem: it is a factory method that creates and returns a new array, using its argument values (regardless of how many of them there are) as the array elements:

```
Array.of() // => []; returns empty array with no arguments
```

```
Array.of(10) // => [10]; can create arrays with a single numeric argument
```

```
Array.of(1,2,3) // => [1, 2, 3]
```

Creating arrays using Array.from() Method

- ✎ **Array.from** is another array factory method introduced in ES6. It expects an iterable or array-like object as its first argument and returns a new array that contains the elements of that object.
- ✎ It is also a simple way to make a copy of an array.

```
let copy = Array.from(original);
```

- ✎ **Array.from()** is also important because it defines a way to make a true-array copy of an array-like object. Array-like objects are non-array objects that have a numeric length property and have values stored with properties whose names happen to be integers. When working with client-side JavaScript, the return values of some web browser methods are array-like, and it can be easier to work with them if you first convert them to true arrays:

```
let truearray = Array.from(arraylike);
```

- ✎ **Array.from()** also accepts an optional second argument. If you pass a function as the second argument, then as the new array is being built, each element from the source object will be passed to the function you specify, and the return value of the function will be stored in the array instead of the original value.

Creating arrays using ... Spread Operator

- ☑ In ES6 and later, you can use the “spread operator,” ..., to include the elements of one array within an array literal:

```
let a = [1, 2, 3];
```

```
let b = [0, ...a, 4]; // b == [0, 1, 2, 3, 4]
```

- ☑ The three dots “spread” the array a so that its elements become elements within the array literal that is being created. It is as if the ...a was replaced by the elements of the array a, listed literally as part of the enclosing array literal. (Note that, although we call

these three dots a spread operator, this is not a true operator because it can only be used in array literals and function invocations.)

- ☑ The spread operator is a convenient way to create a (shallow) copy of an array:

```
let original = [1,2,3];
```

```
let copy = [...original];
```

```
copy[0] = 0; // Modifying the copy does not change the original
```

```
original[0] // => 1
```

- ☑ The spread operator works on any iterable object. Strings are iterable, so an easy way to remove duplicate elements from an array is to convert the array to a set and then immediately convert the set back to an array using the spread operator:

```
let letters = [..."hello world"];
```

```
[...new Set(letters)] // => ["h","e","l","o"," ","w","r","d"]
```

Example:

```
let arr1 = [1, 2, 3];
```

```
let arr2 = ["apple", "banana", "orange"];
```

```
// creates a new array with six elements:
```

```
//[1, 2, 3, "apple", "banana", "orange"]
```

```
let newArr = [...arr1, ...arr2];
```

Accessing JavaScript Arrays

- ☑ You can access an array element using the [] operator following a reference to the array. Inside the brackets, give the index of the element, which should be a non-negative integer. This way you can use for both reading and writing elements.

```
let myArray = [];

myArray[0] = 1; // write first element of myArray

myArray[1] = 2.7 // write second element of myArray

myArray[2] = "gamma"; // write third element of myArray

let i = 3;

myArray[i] = true; // write fourth element of myArray
```

- ☑ You can also write values to an array when you create it:

```
let secondArray = [1, 4.5, false, "y"];
```

- ☑ To read the value of an array element, specify its index. Indexes are integers and the first element as index 0

```
var collection ["blue", "red", 7, false];
collection[0]; // blue
collection[2]; // 7
collection[3]; // false
```

Indexes and properties in JS Array

- Arrays are specialised form of object and you can use square brackets also to access the array's properties.
- Internally JavaScripts converts indexes to string properties, so the index 0 becomes "0", the index 1 becomes "1" etc.

- For indexes that are non-negative integers it does not matter, they are still indexes. But if you put in square brackets non-integer value as an array index and assign a value to it, new property of the array will be created instead of an array element.

```
let array = [];
array[0] = 1; // array element
array[1] = 2; // array element
array[1.5] = 3; //array property
array['d'] = 4; // array property
```

- If index is string that is integer or it is floating point value it is treated as equivalent of integer index.
- When you refer to not existing element there is no error, but undefined is returned.

```
let smallArray = [10, 20, 30];
console.log(smallArray[5]); //undefined
```

Array Length in JS

- ☑ The array has a **length** property. It returns the value which is the **highest index plus one**. If there are no gaps in the array's indexes, the length specifies the number of elements in the array.

```
let array = ['x', 'y', 'z'];
array.length; // 3
```

- ☑ If there are gaps in the array's indexes, the length is greater than the number of elements.

```
let array = [];
array[5] = 'x';
array.length; // 6
```

- ☑ You can also set the length of the array. If it is bigger than the previous one, it does not affect the list of elements. If the new length it is smaller than the previous one, all elements whose indexes are greater than or equal to current length are deleted.

```
let array = [];
array.length = 4; // no new elements are added, it is equivalent of
new Array(4)
```

```
let array = [2, 4, 6, 8, 10];
array.length = 3; // new length is 3, it removed the last 2
elements
```

Adding and Deleting Array Elements

- ☑ The simplest way to add elements to an array: **just assign values to new indexes.**

```
let a = []; // Start with an empty array.
```

```
a[0] = "zero"; // And add elements to it.
```

```
a[1] = "one";
```

- ☑ You can also use the **push() method** to add one or more values to the end of an array:

```
let a = []; // Start with an empty array
```

```
a.push("zero"); // Add a value at the end. a = ["zero"]
```

```
a.push("one", "two"); // Add two more values. a = ["zero", "one", "two"]
```

- ☑ Pushing a value onto an array `a` is the same as assigning the value to `a[a.length]`
- ☑ To delete an array element you can use the delete operator. This operation does not shift the array's indexes and does not change the length of the array. In the position of the removed element of the array, a gap is left(sparse array is formed)

```
let array = [2, 4, 6];
delete array[1];
array[1]; // undefined
array.length; // 3
```

- ☑ The **pop()** method removes (pops) the last element of an array. The pop() method changes the original array. This returns the removed element.

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
let removed = fruits.pop();//Mango
//now array becomes
// fruits = ["Banana", "Orange", "Apple"]
```

- ☑ The **shift()** method removes the first item of an array. The shift() method changes the original array. This returns the shifted element.

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];

let removed = fruits.shift();//Banana is removed

//now array becomes

// fruits = ["Banana", "Orange", "Apple", "Mango"]
```

- ☑ The **unshift()** method adds new elements to the beginning of an array. The unshift() method overwrites the original array.

Syntax: array.unshift(item1, item2, ..., itemN)

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];

fruits.unshift("Lemon", "Pineapple");

// ["Lemon", "Pineapple", "Banana", "Orange", "Apple", "Mango"]
```

Multidimensional Arrays

- ☑ JavaScript does not have true multidimensional arrays. But you can create the equivalent as array of arrays. To access the array element, use the appropriate number of times the [] operator and enter the appropriate indexes

```
let multi = [];
multi[0] = ['a', 'b', 'c'];
multi[1] = ['x', 'y', 'z'];
console.log(multi[1][2]); // z

let matrix = new Array(3);
matrix[0] = new Array(1, 2);
matrix[1] = new Array(10, 20);
matrix[2] = new Array(100, 200);
console.log(matrix[2][1]); // 200
```

Typed Arrays (explore yourself)

Iterating Arrays

- ☑ To access and process all elements of the array an iteration is used.
- ☑ One of the ways to iterate an array is through the for loop

```
var flowers = ['rose', 'tulip', 'lily', 'aster'];
for (var i = 0; i < flowers.length; i++) {
    console.log(flowers[i]);
}
// Output:
// rose
// tulip
// lily
// aster
```

Note that an array may have gaps, then in your code you should handle nonexistent elements.

- ☑ Second way is to use for-in loop. It iterates only existing elements. In fact it iterates over all enumerable properties, also inherited from **Array.prototype**. It is advisable not to use this loop to iterate arrays unless you filter redundant items.

```
let fruits = ['apple', 'strawberry', 'cherry'];
fruits[5] = 'pear';
for (let index in fruits) {
    if (fruits.hasOwnProperty(index)) {
        console.log(fruits[index]);
    }
}
//Output:
// apple
// strawberry
// cherry
// pear
```

- ☑ Another way of iterating arrays is the **forEach** method. It iterates array elements in index order and each one passes to a function passed as an argument.

```
let trees = ['brich', 'pine', 'oak'];
trees.forEach(function(tree) {
    console.log('tree: ' + tree);
});
// Output:
// tree: brich
// tree: pine
// tree: oak
```

- ☑ New added in ECMAScript 2015 the **for-of loop** is another choice. It does not iterate over indexes like the for-in but over values.

```
let vegetables = ['tomato', 'cucumber', 'carrot'];
for (var vegetable of vegetables) {
    console.log(vegetable);
}
// Output:
// tomato
// cucumber
// carrot
```

JavaScript Array and forEach()

- ☑ The forEach() method calls a function for each element in an array.
- ☑ The forEach() method is not executed for empty elements.

Syntax:

forEach(function(currentValue, index, arr), thisValue)

function()	Required. A function to run for each array element.
currentValue	Required. The value of the current element.
index	Optional. The index of the current element.
arr	Optional. The array of the current element.

thisValue Optional.
 Default undefined.
 A value passed to the
 function as its this value.

Example:

```
<html>

<body>

<h1>JavaScript Arrays</h1>

<h2>The forEach() Method</h2>

<p>forEach() calls a function for each element in an array:</p>

<p>Compute the sum of the values in an array:</p>

<p id="demo"></p>

<script>

let sum = 0;

const numbers = [65, 44, 12, 4];

numbers.forEach(myFunction);

document.getElementById("demo").innerHTML = sum;

function myFunction(item) {

    sum += item;

}
```

```
</script>
```

```
</body>
```

```
</html>
```

JS for- of loop

- ☑ A for-of loop in JavaScript is used to iterate over the values of an iterable object such as an array, a string, or a map. Here's the basic syntax for a for-of loop in JavaScript:

```
for (value of iterable) {  
  
    // code to be executed  
  
}
```

- The *value* variable represents the current value of the iteration, and the *iterable* variable represents the iterable object.
- The loop will iterate over each *value* in the *iterable* object, executing the code within the loop for each one.

Example:

```
const myArray = [1, 2, 3, 4, 5];  
  
for (let myValue of myArray) {  
  
    console.log(myValue);  
  
}
```

- This loop will iterate over the values of the array and print out each value. In this case, the loop will print the following output:

1
2
3
4
5

Another Example

```
<html>

  <body>

    <h2>JavaScript For Of Loop</h2>

    <p>The for of statement loops through the values of an
    iterable object.</p>

    <p id="demo"></p>

    <script>

      let language = "JavaScript";

      let text = "";

      for (let x of language) {

        text += x + "<br>";

      }

      document.getElementById("demo").innerHTML = text;

    </script>
```

</body>

</html>

Array methods

- In JavaScript, there are various array methods available that makes it easier to perform useful operations. Some of them are listed below:

Methods	Description
concat()	It returns a new array object that contains two or more merged arrays.
copywithin()	It copies the part of the given array with its own elements and returns the modified array.
entries()	It creates an iterator object and a loop that iterates over each key/value pair.
every()	It determines whether all the elements of an array are satisfying the provided function conditions.
flat()	It creates a new array carrying sub-array elements concatenated recursively till the specified depth.
flatMap()	It maps all array elements via mapping function, then flattens the result into a new array.
fill()	It fills elements into an array with static values.
from()	It creates a new array carrying the exact copy of another array element.
filter()	It returns the new array containing the elements that pass the provided function conditions.
find()	It returns the value of the first element in the given array that satisfies the specified condition.
findIndex()	It returns the index value of the first element in the given array that satisfies the specified condition.
forEach()	It invokes the provided function once for each element of an array.
includes()	It checks whether the given array contains the specified element.
indexOf()	It searches the specified element in the given array and returns the index of the first match.
isArray()	It tests if the passed value is an array.
join()	It joins the elements of an array as a string.

keys()	It creates an iterator object that contains only the keys of the array, then loops through these keys.
lastIndexOf()	It searches the specified element in the given array and returns the index of the last match.
map()	It calls the specified function for every array element and returns the new array
of()	It creates a new array from a variable number of arguments, holding any type of argument.
pop()	It removes and returns the last element of an array.
push()	It adds one or more elements to the end of an array.
reverse()	It reverses the elements of given array.
reduce(function, initial)	It executes a provided function for each value from left to right and reduces the array to a single value.
reduceRight()	It executes a provided function for each value from right to left and reduces the array to a single value.
some()	It determines if any element of the array passes the test of the implemented function.
shift()	It removes and returns the first element of an array.
slice()	It returns a new array containing the copy of the part of the given array.
sort()	It returns the element of the given array in a sorted order.
splice()	It add/remove elements to/from the given array.
toLocaleString()	It returns a string containing all the elements of a specified array.
toString()	It converts the elements of a specified array into string form, without affecting the original array.
unshift()	It adds one or more elements in the beginning of the given array.
values()	It creates a new iterator object carrying values for each index in the array.

concat() method:

- ☑ The concat() method concatenates (joins) two or more arrays.
- ☑ The concat() method returns a new array, containing the joined arrays.
- ☑ The concat() method does not change the existing arrays.

Syntax: `array1.concat(array2, array3, ..., arrayN)`

```
let a = [3,5,6,7];
let b = [10,12];
let c = [1.5,4.6,8.9,9.5];
```

```
let d = a.concat(b,c);//[3,5,6,7,10,12,1.5,4.6,8.9,9.5]
```

fill() method:

- ☑ The **fill()** method fills specified elements in an array with a value.
- ☑ The fill() method overwrites the original array.
- ☑ Start and end position can be specified. If not, all elements will be filled.

Syntax: *array.fill(value, start, end)*

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
fruits.fill("Kiwi");//["Kiwi", " Kiwi ", " Kiwi ", " Kiwi "]
```

```
fruits.fill("Kiwi", 1, 3);//["Banana", "Kiwi", "Kiwi", "Mango"]
```

filter Method:

- ☑ The **filter()** method creates a new array filled with elements that pass a test provided by a function.
- ☑ The filter() method does not execute the function for empty elements.
- ☑ The filter() method does not change the original array.

Syntax: *array.filter(function(currentValue, index, arr), thisValue)*

Parameter	Description
function()	Required. A function to run for each array element.
currentValue	Required. The value of the current element.
index	Optional. The index of the current element.
arr	Optional. The array of the current element.
thisValue	Optional. Default undefined .A value passed to the function as its this value.

```
let marks=[50,40,45,37,20];

function check(value)
{
  return value>30;
}
let marksMoreThan30 = marks.filter(check);
```

find() Method:

- ☑ The **find()** method returns the value of the first element that passes a test.
- ☑ The find() method executes a function for each array element.
- ☑ The find() method returns undefined if no elements are found.
- ☑ The find() method does not execute the function for empty elements.
- ☑ The find() method does not change the original array.

Syntax:

```
array.find(function(currentValue, index, arr),thisValue)
```

Parameter	Description
function()	Required. A function to run for each array element.
currentValue	Required. The value of the current element.
index	Optional. The index of the current element.

arr	Optional. The array of the current element.
thisValue	Optional.Default undefined. A value passed to the function as its this value.

```
const ages = [3, 10, 18, 20,24];
```

```
function checkAge(age) {
  return age > 18;
}
```

```
let a = ages.find(checkAge);// 20
```

```
const array1 = [5, 12, 8, 130, 44];
```

```
const found = array1.find(element => element > 10);
```

```
console.log(found);//12
```

join() Method

- ☞ The **join()** method returns an array as a string.
- ☞ The join() method does not change the original array.
- ☞ Any separator can be specified. The default is comma (,)

Syntax: *array.join(separator)*

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
let text = fruits.join();
console.log(text); // Banana,Orange,Apple,Mango
Banana/Orange/Apple/Mango
```

```
let text2 = fruits.join('/');
console.log(text2); // Banana/Orange/Apple/Mango
```

includes() Method

- ☞ The **includes()** method returns true if an array contains a specified value.
- ☞ The includes() method returns false if the value is not found.
- ☞ The includes() method is case sensitive.
- ☞ Syntax: `array.includes(element, start)`
//start is where to start the search (0 by default)

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];

let result = fruits.includes("Mango");//true

let result2 = fruits.includes("Banana",2);//false
```

indexOf() Method

- ☞ The **indexOf()** method returns the first index (position) of a specified value.
- ☞ The indexOf() method returns -1 if the value is not found.
- ☞ The indexOf() method starts at a specified index and searches from left to right.
- ☞ By default the search starts at the first element and ends at the last.
- ☞ Negative start values counts from the last element (but still searches from left to right).
- Syntax: `array.indexOf(item, start)`
//start is where to start the search (0 by default)

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
let result = fruits.indexOf("Apple");//2
```

```
let result2 = fruits.indexOf("Banana",2);//-1
```

Defining and Invoking Functions

- ☑ A function is a block of JavaScript code that is defined once but may be executed, or invoked, any number of times.
- ☑ Functions in JavaScript allow you to extract piece of code, give it a name and reference to it wherever it is needed.
- ☑ JavaScript functions are **parameterized**: a function definition may include a list of identifiers, known as **parameters**, that work as local variables for the body of the function.
- ☑ Function invocations provide values, or **arguments**, for the function's parameters.
- ☑ Functions often use their argument values to compute a **return** value that becomes the value of the *function-invocation* expression.
- ☑ In addition to the arguments, each invocation has another value - the invocation context that is the value of the **this** keyword.
- ☑ Functions are a fundamental building block for JavaScript programs and a common feature in almost all programming languages

NOTES:

1. If a function is assigned to a property of an object, it is known as a **method** of that object. When a function is invoked on or through an object, that object is the **invocation context** or *this* value for the function. Functions designed to initialize a newly created object are called **constructors**.
2. In JavaScript, **functions are objects**, and they can be manipulated by programs. JavaScript can assign functions to variables and pass them to other functions, for example. Since functions are objects, you can set properties on them and even invoke methods on them.

Function definition

- ☑ JavaScript function can be defined using a **function declaration statement** or as a **function definition expression**.

Function declaration statement

- ☑ In **function declaration statement**, function definition begins with the **function keyword** followed by the **function name**, **pair of round brackets** inside of which there is a list of zero or more **parameters** and **pair of curly brackets** with zero or more JavaScript statements inside.

- ☞ Function declaration statement is like declaration of a variable and assigning a function object to it.
- ☞ Function name can be any valid JavaScript identifier. Then you can use this name to refer to that function in the code.

Example:

```
// to display info message on alert
function info(name, surname) {
    let message = "Hello " + name + " " +
surname;
    alert(message);
}
```

More examples :

// Compute the distance between Cartesian points (x1,y1) and (x2,y2).

```
function distance(x1, y1, x2, y2) {

    let dx = x2 - x1;

    let dy = y2 - y1;

    return Math.sqrt(dx*dx + dy*dy);

}
```

// A recursive function (one that calls itself) that computes factorials

// Recall that x! is the product of x and all positive integers less than it.

```
function factorial(x) {
    if (x <= 1) return 1;
    return x * factorial(x-1);
}
```

Function definition expression (Function expression)

- In this case a function is defined as a part of an expression. Function definition also begins with the function keyword. Function name is optional, the rest is the same as in the function declaration.
- Function expressions look a lot like function declarations, but they appear within the context of a larger expression or statement, and the name is optional.

```
const info = function (name, surname) {
    let message = "Hello " + name + " " + surname;
    alert(message);
};
```

- Function defined using function declaration expression can be assigned to a variable or can be used only once in an expression.
- If you assign function to a variable, at the end of the expression put a semicolon. You can use this variable to refer to that function in the code.
- It is a good practice to use const with function expressions so you don't accidentally overwrite your functions by assigning new values.

Here are some example function expressions:

// This function expression defines a function that squares its argument.

// Note that we assign it to a variable

```
const square = function(x) { return x*x; };
```

// Function expressions can include names, which is useful for recursion.

```
const f = function fact(x) {
    if (x <= 1) return 1;
    else return x*fact(x-1);
};
```

// Function expressions can also be used as arguments to other functions:

```
[3,2,1].sort(function(a,b) {
    return a-b;
});
```

// Function expressions are sometimes defined and immediately invoked:

```
let tensquared = (function(x) {return x*x;})(10);
```

Anonymous functions

- ☑ Functions used as a function expression without a name are called anonymous functions.

```
let show = function () {
    console.log('Anonymous function');
};
```

- Anonymous functions are functions without names.

- Anonymous functions can be used as an argument to other functions or as an immediately invoked function execution.

Scope of functions

- ☞ Functions defined both ways have different scope.
- ☞ Functions defined using declaration statement are hoisted to the top of the script or enclosing function. Therefore they can be invoked in the code above the place, they are declared.
- ☞ Functions defined as expression cannot be invoked before they are defined. Assignments to variables are not hoisted.

Return statement in JS function

- Functions not only execute a code defined inside their body but also can return value by using the return statement.
- If the return statement has an associated expression, value of this expression is returned. Otherwise it returns the undefined value.
- Also if a function does not have the return statement it returns the undefined value
- Note that using the return statement it is possible early termination of function, for example if the condition is not met.

```
function calculateBmi(weight, height) {  
  
    if (!weight || !height) {  
  
        return;  
  
    }  
  
    var bmi = weight / height * height;
```

```
        return bmi;
    }
}
```

Nested functions

- Functions in JavaScript can be nested inside other functions.

```
function calculations(val1, val2, val3) {

    function multiply(val1, val2) {

        return val1 * val2;

    }

    var result = multiply(val1, val2) + multiply(val1, val3) +
multiply(val2, val3);

    return result;

}
```

NOTES

→ As you have seen in the previous examples, JavaScript functions are defined with the function keyword. Functions can also be defined with a built-in JavaScript function constructor called `Function()`.

```
const myFunction = new Function("a", "b", "return a * b");  
let x = myFunction(4, 3);
```

→ **Functions are Objects**

- The *typeof* operator in JavaScript returns "function" for functions.
- But, JavaScript functions can best be described as objects.
- JavaScript functions have both properties and methods.
- The *arguments.length* property returns the number of arguments received when the function was invoked
- The *toString()* method returns the function as a string

Invoking Functions

- The JavaScript code that makes up the body of a function is not executed when the function is defined, but rather when it is invoked.
- Function invocation refers to the process of calling or executing a JavaScript function.
- JavaScript functions can be invoked in several ways:
 - **As functions**
 - **As methods**
 - **As constructors**
 - **Indirectly through their call() and apply() methods**
 - **Implicitly, via JavaScript language features that do not appear like normal function invocations**

Function invocation

- ☞ To invoke a function specify the function providing its name or a variable to which the function was assigned. Then add a pair of round brackets inside which put a comma-separated list of zero or more arguments.

```
info('Bipin', 'Timalsina');
```

```
let amount = calculations(2, 4, 5);
```

- ☞ Values passed as **arguments** are assigned to the function **parameters**. Then the code in the body of the function is executed. The value of the return statement is the value of the invocation expression.

Method invocation

- ☞ A method is a function stored in a property of a JavaScript object.

```
let myObject = {};
```

```
myObject.func = function (message) {

    alert('User sent a message: ' + message);

}
```

- ☞ To invoke a method refer to the object followed by dot “.”, followed by the method name and a pair of round brackets and list of zero or more arguments inside them.

```
myObject.func('Namaskar!');
```

- ✎ Method invocation works in the same way as function invocation. The main difference is the invocation context. The context for a method is the object that calls it. In the function body you can refer to that object using the keyword *this*.

```
let calculation = {

    factor: 2,

    multiply: function(value) {

        return value * this.factor;

    }

};

let result = calculation.multiply(5); // returns 10
```

Method chaining

- ✎ Method invocation uses property access expression and may be more complex.
- ✎ An object can contain another object whose method you want to call:

```
vehicle.engine.start();
```

- ✎ You can invoke a method that returns an object, then you can directly invoke another method on it

```
store.getItem('camera').deduct(1);
```

- ✎ In this way you can invoke chain of methods as a single expression.

Constructor invocation

- ✎ Functions may be used to create objects. If a function is preceded by the keyword `new` then it is a constructor invocation.

```
let myObject = new MyObject();
```

- ✎ If a constructor does not have parameters, then you can omit parentheses.

```
let myObject = new MyObject;
```

- ✎ A constructor invocation creates a new object and the constructor function initializes it. The new object is the invocation context and constructor can refer to it using the *this* keyword.
- ✎ Constructor functions usually do not use the *return* keyword. They initialize a new object and this object is the value of the constructor invocation expression.
- ✎ If a constructor explicitly returns an object, then that object is the value of the constructor invocation. If it returns a primitive value, this value is ignored and the new object is the value of the invocation.

```
function Person(name, age) {  
  
    this.name = name;
```

```

    this.age = age;

}

const p1 = new Person("John", 30);

console.log(p1.name); // Output: John

```

Indirect Invocation (using `call()` and `apply()` methods)

- ✎ JavaScript functions are objects, and like all JavaScript objects, they have methods. Two of these methods, **`call()`** and **`apply()`**, invoke the function indirectly.
- ✎ Indirect function invocation refers to calling a function through a reference to the function rather than calling the function directly by its name. This can be done using the **`call()`** or **`apply()`** method
- ✎ **`call()`** and **`apply()`** allow you to indirectly invoke a function as if it were a method of some other object.
- ✎ Both methods allow you to explicitly specify the *this* value for the invocation, which means you can invoke any function as a method of any object, even if it is not actually a method of that object.
- ✎ Both methods also allow you to specify the arguments for the invocation.
- ✎ The first argument to both **`call()`** and **`apply()`** is the object on which the function is to be invoked; this argument is the invocation context and becomes the value of the *this* keyword within the body of the function.
- ✎ The `call()` method uses its own argument list as arguments to the function, and the `apply()` method expects an array of values to be used as arguments

```

//function invocation using call()

function greet() {

    console.log("Hello, " + this.name + "!");

}

```

```
const person = { name: "John" };

greet.call(person); // Output: Hello, John!
```

In this example, the **call()** method is used to call the **greet** function with the **person** object as the **this** value. This allows us to use the name property of the person object within the function.

```
// function invocation using apply()

function sum(a, b, c) {

    return a + b + c;

}

const numbers = [1, 2, 3];

console.log(sum.apply(null, numbers)); // Output: 6
```

In this example, the **apply()** method is used to call the sum function with an array of numbers as the arguments. The first argument of the **apply()** method is set to *null*, which sets the **this** value to the global object.

Another example of using the **apply()** method is to call a function with an object as the **this** value:

```
const person = {

    name: "John",

    greet: function() {

        console.log("Hello, " + this.name + "!");

    }

}
```

```

    }

};

const anotherPerson = { name: "Jane" };

person.greet.apply(anotherPerson); // Output: Hello, Jane!

```

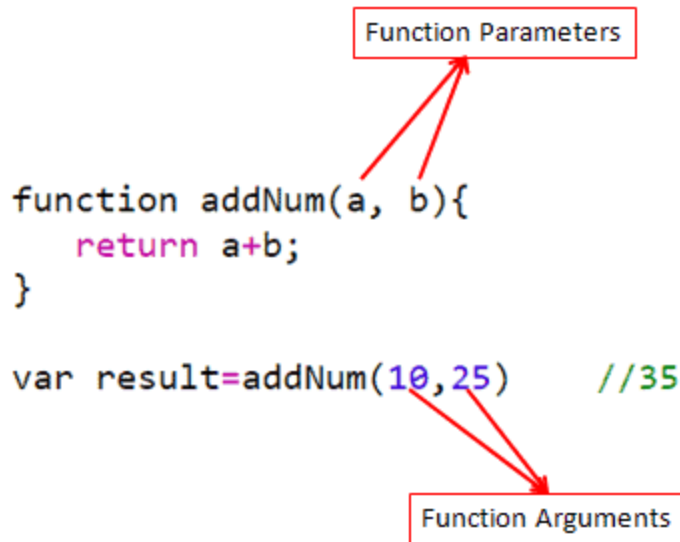
In this example, the **apply()** method is used to call the **greet** method of the **person** object with the **anotherPerson** object as the *this* value. This allows us to use the **name** property of the **anotherPerson** object within the function.

Implicit Function Invocation

- ✎ There are various JavaScript language features that do not look like function invocations but that cause functions to be invoked
- ✎ Some of the language features that can cause implicit function invocation :
 - If an object has getters or setters defined, then querying or setting the value of its properties may invoke those methods
 - When an object is used in a string context (such as when it is concatenated with a string), its **toString()** method is called. Similarly, when an object is used in a numeric context, its **valueOf()** method is invoked
 - When you loop over the elements of an iterable object, there are a number of method calls that occur

Function parameters and arguments

- ☑ Function can be parameterized. It means that in the definition of a function you define parameters in parentheses and use them in the function body:
- ☑ Values passed into an invoked function are called arguments



- ☑ We can define a JavaScript function with zero to any number of parameters
- ☑ We can use any JavaScript value like numbers, strings, objects as arguments
- ☑ Parameters are evaluated from left to right
- ☑ Function definition does not specify an expected type for its parameters.
- ☑ Function invocation even does not check if the number of passed arguments fits the number of parameters in function definition. (The number of arguments does not have to match the number of parameters)
- ☑ If the number of passed arguments is smaller than defined parameters, then parameters for missing arguments are set to the default value usually *undefined*.

```

function myLog(a, b) {
    console.log('val a: ' + a);
    console.log('val b: ' + b);
}

```

```
myLog('ok');
```

```

// Output:
// val a: ok
// val b: undefined

```

Default function parameters

- ☑ We can also configure parameters with default values (known as **default parameter**) while declaring the function using an assignment operator

```
function power(x, n = 2) {  
  
    result = 1;  
  
    for (let i = 1; i <= n; i++) {  
  
        result = result * x;  
  
    }  
  
    return result;  
  
}  
  
console.log(power(2, 3)); // 8  
  
console.log(power(2)); // 4
```

The arguments object

- ☑ The arguments object allows us to read all passed arguments in the function invocation.
- ☑ This is useful when the number of passed arguments differs from the number of defined function parameters, especially when it is bigger.
- ☑ It is way to define vararg functions.
- ☑ The arguments object is like an array and contains a numerical list of arguments in the order in which they were passed.

```
function add(x, y) {  
  
    var result = 0;  
  
    if (arguments.length == 2) {
```

```

        result = x + y;

    } else {

        for (let i = 0; i < arguments.length; i++) {

            result = result + arguments[i];

        }

    }

    return result;

}

var result = add(3, 5); // 8

var result2 = add(3, 5, 6); //14

```

- ☑ If a function has defined two parameters x and y and you pass three arguments, then you can read two first arguments using parameters names x and y or using arguments[0] and arguments[1]. The third argument is accessible only as arguments[2].

Rest parameters

- ☑ In JavaScript, a rest parameter is a way to represent an indefinite number of arguments as an array within a function's parameter list. The rest parameter is denoted by an ellipsis (...) followed by a parameter name, which collects all remaining arguments passed to the function into an array.
- ☑ A rest parameter is preceded by three periods, and it must be the last parameter in a function declaration.

- ☑ When you invoke a function with a rest parameter, the arguments you pass are first assigned to the non-rest parameters, and then any remaining arguments (i.e., the “rest” of the arguments) are stored in an array that becomes the value of the rest parameter. This
- ☑ Functions like the previous example that can accept any number of arguments are called **variadic** functions, **variable arity** functions, or **vararg** functions
- ☑ Syntax:

```
function functionName(a, b, ...theArgs) {
    //function body
}
```

Example

```
function sum(...numbers) {

    let total = 0;

    for (let number of numbers) {

        total += number;

    }

    return total;

}

sum(1, 2, 3, 4, 5); // returns 15
```

In this example, the sum function takes any number of arguments and uses a rest parameter named numbers to collect all the arguments into an array. The function then iterates through the array and adds up all the numbers, returning the total.

- ☑ Rest parameters can be used with any number of arguments, including none. In the case where no arguments are passed to the function, the rest parameter will simply be an empty array.

- ☑ Rest parameters are especially useful in situations where you need to pass a variable number of arguments to a function, such as when working with arrays or other data structures. They can also be used with the spread operator (...) to concatenate arrays or other iterables.
- ☑ It's worth noting that rest parameters must be the last parameter in a function's parameter list, as they collect all remaining arguments passed to the function.

Arguments are passed by value

- Arguments in JavaScript are passed to function by value. It means that argument is copied into parameter variable. Consider following example:

```
function increment(val) {
    val = val + 1;
}
```

```
var size = 3;
increment(size);
console.log(size); // 3
```

In case of code execution above value of size is copied into parameter val and then val is incremented. However value of size does not change and remains equal to 3.

NOTE: Variables defined outside of a function are global and are visible throughout JavaScript program. Whereas variables defined inside a function are local and are not accessible outside this function.

Arrow functions

```
const add =function(a, b) {
```

```

    return a + b;
}
const add = (a, b) => a + b;

```

- ☑ Arrow functions are a shorter syntax for writing function expressions. They use the => token and make the code more concise.
- ☑ Arrow functions are anonymous and do not have its own *this*, arguments and super.
- ☑ They are best suited as one-line functions and cannot be used as constructors
- ☑ Arrow functions were introduced in ES6.
- ☑ The function keyword is not used, and, since arrow functions are expressions instead of statements, there is no need for a function name, either.
- ☑ The general form of an arrow function is a comma-separated list of parameters in parentheses, followed by the => arrow, followed by the function body in curly braces:
- ☑ Syntax:

(param1 [, paramN]) => { statements }

Example:

```
const sum = (x, y) => { return x + y; };
```

- ✎ Parentheses are optional if a function has only one parameter:

param => { statements }

Example:

```
const polynomial = x => {return x*x + 2*x + 3};
```

- ✎ If a function has no parameters you should use a pair of parentheses

() => { statements }

Example:

```
const fun = () => {
```

```

let a = 5;

console.log('Hello! Value of a is'+a);

};

fun(); // Hello! Value of a is 5

```

- ✎ If the body of the function is a single return statement, you can omit the return keyword, the semicolon that goes with it, and the curly braces, and write the body of the function as the expression whose value is to be returned:

```
const sum = (x, y) => x + y;
```

- ✎ Arrow functions are a more concise way to define functions and are especially useful for writing callbacks and event handlers

Objects in JS

Objects in programming language are a certain approximation of objects in the real world, for example wallet, car, telephone, etc. Each object in real world has its own state and behavior. In programming language state of the object is stored in fields and its behavior is expressed through methods.

- **JavaScript object** is unordered collection of properties, where each property has a name and value. A property's value can be a primitive value, object or function. In the last case the property is called a method.

```

let person = {
  firstName: "Bipin",
  lastName: "Timalina",
  age: 30,
  gender: "male",

```

```

    fullName: function(){
        return this.firstName + " " + this.lastName;
    }
};

```

In the example above, *person* is an object that has five properties: *firstName*, *lastName*, *age*, *gender* and *fullName*, with their respective values. Here, *fullName* is a method that returns full name using the *firstName* and *lastName* property of the object. The *this* keyword refers to the object itself, so *this.firstName* refers to the *firstName* property of the object.

- We can distinguish three categories of objects:
 1. **Native objects** are instances of object types and objects defined by the ECMAScript specification (eg. Object, Array, Date, Math),
 2. **Host objects** are objects defined by host environment like a web browser (eg. document, Node)
 3. **Objects defined by users.**
- An object in JS is a composite value: it aggregates multiple values (primitive values or other objects) and allows you to store and retrieve those values by name.
- In addition to maintaining its own set of properties, a JavaScript object also inherits the properties of another object, known as its “prototype.” The methods of an object are typically inherited properties, and this “prototypal inheritance” is a key feature of JavaScript.
- JavaScript objects are dynamic— properties can usually be added and deleted—but they can be used to simulate the static objects and “structs” of statically typed languages.

NOTE: Any **value** in JavaScript that **is not** a *string*, a *number*, a *Symbol*, or *true*, *false*, *null*, or *undefined* **is an object**. And **even though strings, numbers, and booleans are not objects, they can behave like immutable objects.**

- **Objects are mutable** and manipulated by **reference** rather than by value. If the variable *x* refers to an object and the code `let y = x;` is executed, the variable *y* holds a reference to

the same object, not a copy of that object. Any modifications made to the object through the variable y are also visible through the variable x.

- The most common things to do with objects are to create them and set, query, delete, test, and enumerate their properties
- A **property** has a *name* and a *value*. A property name may be any string, including the empty string (or any Symbol), but no object may have two properties with the same name. The value may be any JavaScript value, or it may be a getter or setter function (or both).
- It is sometimes important to be able to distinguish between properties defined directly on an object and those that are inherited from a prototype object. JavaScript uses the term **own property** to refer to non-inherited properties.
- In addition to its *name* and *value*, each **property** has three property attributes:
 - The **writable** attribute specifies whether the value of the property can be set.
 - The **enumerable** attribute specifies whether the property name is returned by a *for/in* loop.
 - The **configurable** attribute specifies whether the property can be deleted and whether its attributes can be altered.

NOTES:

- ☞ Many of JavaScript's **built-in objects** have properties that are *read-only*, *nonenumerable*, or *non-configurable*.
- ☞ By default, however, all properties of the **objects you create** are *writable*, *enumerable*, and *configurable*

Creating Objects in JS

- ☞ There are several ways to create objects in JavaScript. Mainly we use following three techniques:
 1. Object creation using **object literals**
 2. Object creation with **new keyword** (using constructor function)
 3. Object creation using **Object.create()** method

Object creation using object literals

- ☞ We can create an object using an object literal which is the simplest way to create an object. An object literal is a comma-separated list of name-value pairs wrapped in curly braces {}.
- ☞ Object can be created with an object literal. An object literal is enclosed in curly braces ({}), a comma-separated list of property name : value pairs.
- ☞ An object literal is an expression and whenever it appears, it creates new, distinguish object. An object created with object literal is like created with new Object() and is an instance of Object.

☞ Example:

```
let user = {    // an object

    name: "John", // by key "name" store value "John"

    age: 30      // by key "age" store value 30

};
```

Some more examples:

```
let empty = {};           // An object with no properties
let point = { x: 0, y: 0 }; // Two numeric properties
let p2 = { x: point.x, y: point.y+1 }; // More complex values
let book = {
    "main title": "JavaScript", // These property names include spaces,
    "sub-title": "The Definitive Guide", // and hyphens, so use string literals.
    for: "all audiences", // for is reserved, but no quotes.
    author: { // The value of this property is
        firstname: "David", // itself an object.
        surname: "Flanagan"
    }
};
```

Creating objects using constructor function(with new keyword)

- To create an object with constructor function we use the **new** keyword followed by a constructor invocation.
- The syntax is as follow:

let obj = new Constructor();

- The **new** operator creates a new object. A function that follows it is a constructor and its task is to initialize the object.
- A constructor function is a regular JavaScript function that is used to initialize an object. It is named with a capital letter to differentiate it from other functions.
- Objects can be created using ***built-in JS constructors*** or ***user-defined constructor functions***.
 - JavaScript provides constructors for native object types like Object, Array, Date. You can use them to create an objects.

```
let o = new Object(); // Create an empty object: same as {}.
let a = new Array();  // Create an empty array: same as [].
let d = new Date();    // Create a Date object representing the current time
let r = new Map();      // Create a Map object for key/value mapping
```

NOTE: The **Object** is a general object type and all other objects inherit properties from it

- We can define our own object type and create instance of it. To do this we have to define a function, specify its name, properties and methods. Such a function is called a constructor.

Example:

```
function Person(name, age, gender) {
  this.name = name;
  this.age = age;
  this.gender = gender;
}

let person1 = new Person("John", 30, "male");
```

In the example above, we defined a constructor function **Person** which takes three parameters *name*, *age*, and *gender*. We use the **new** keyword to create a new instance of the *Person* object and assign it to the variable *person1*.

Object creation using Object.create() method

- Each object in JavaScript inherits properties from some other object. This object is called prototype. Rarely, the prototype of the object can be set to null. Objects created by object literal have **Object.prototype** prototype. All instances of built-in object types also inherit from **Object.prototype** prototype.
- Object.create()** function allows you to create a new object and specify in the first argument the prototype of that object.

```
let obj = Object.create(Object.prototype); // equivalent of { } or new Object()
```

```
let o1 = Object.create({x: 1, y: 2});    // o1 inherits properties x and y.
o1.x + o1.y                               // => 3
```

- You can pass null to create a new object that does not have a prototype, but if you do this, the newly created object will not inherit anything, not even basic methods like `toString()` (which means it won't work with the `+` operator either):

```
let o2 = Object.create(null);             // o2 inherits no props or methods.
```

Querying and Setting Properties in JS Object

- State of the object is stored in its fields and behavior in its methods. These fields and methods are called object **properties**. A property of an object is similar to a variable although they are not the same. Property is enclosed in the object, has a name, a value and also some other features. The properties specify characteristic of the object.

Access to properties

- To access an object property use an object identifier followed by dot (.) and property name.

```
let value = objectName.propertyName;
```

- To add property to the object or update its value use the same notation as you would to access it and assign to it a value.

```
let address = {district: "Kathmandu"};
```

```
address.district = "Lalitpur";
```

```
let address.municipality = "Bagmati";
```

- The second way to get access to the object's property is to use square brackets ([]). Specify an object identifier followed property name enclosed in square brackets.

```
let value = objectName["propertyName"];
```

```
let address = {district: "Kathmandu"};
```

```
address["district"] = "Lalitpur";
```

```
let address["municipality"] = "Bagmati";
```

- Objects in JavaScript are similar to associative arrays. Property name is like string key, using it with the square brackets notation you can get access to the property value. Another

similarity is that objects do not have a fixed number of properties like in some other programming languages. A new properties can be added to the object at any time.

```
let author = book.author;      // Get the "author" property of the book.  
let name = author.surname;     // Get the "surname" property of the author.  
let title = book["main title"]; // Get the "main title" property of the book.
```

NOTES:

Array indexed by strings rather than by numbers is known as an **associative array** (or hash or map or dictionary)

Property name can be any valid string, even the empty string. With dot (.) notation you can access only valid JavaScript identifiers. If you want to use property name that is not a valid identifier, you should use square bracket notation.

```
var obj = {};  
obj['Jack Smith'] = 28;  
obj['012hash'] = 1234;  
obj[''] = 0.01;
```

Checking properties in JS Object

- Sometimes it is useful to check if an object has given property. JavaScript provides several possibilities to do this: by querying the property, the in operator, the hasOwnProperty() method.
- The simplest way is to query the property and check whether it is not undefined.

```
let building = {room : {name : 'living room'}};
```

```
if (building.room !== undefined) // true, building has a property room
```

```
if (building.alarm !== undefined) // false, building does not have a property alarm
```

The **in** operator is more convenient than querying the property. Additionally it allows you to check whether property exists even if it is set to undefined. It returns true if the object has own property with the given name or inherits it. The in operator expects property name on its left and a tested object on the right

```
let building = {room : 'kitchen', ventilation : undefined};

if ('room' in building) // true, building has a property room

if ('ventilation' in building) // true, building has a property ventilation

if ('lighting' in building) // false, building does not have a property lighting
```

The **hasOwnProperty()** method returns true if the object has own property with given name and does not include inherited properties.

```
var building = {room : 'cellar'};

if (building.hasOwnProperty('room')) // true

if (building.hasOwnProperty('heating')) // false

if (building.hasOwnProperty('toString')) // false, toString property is inherited
```

Deleting properties from a JS Object

- It is possible to remove a property from an object.
- To do this, use the delete operator and as an operand, specify the property of the object.
- In this way you can delete only own properties, not inherited.

```
let data = {name : 'Max Doe', age : 33, salary : 3000};

delete data.age;

delete data['salary'];
```

Getters and setters

- JavaScript has the functionality of getters and setters.
- You can define properties known as accessor properties which instead of value contain a combination of getter function and setter function.
- When an accessor property is read, the getter function is called. It should return a proper value and before it can execute some additional statements.
- When an accessor property is set, the setter function is called. It should set a new value and can execute some additional statements.
- To create getter and setter in the object literal the syntax is as follow:

```
let obj = {  
  
    ordinaryProperty : 'value',  
  
    get accessorProperty() {  
  
        // additional statements  
  
        return this.ordinaryProperty;  
  
    },  
  
    set accessorProperty(value) {  
  
        this.ordinaryProperty = value;  
  
        // additional statements  
  
    }  
  
};
```

- Accessor properties are defined as functions with the same name as the property. Instead of the function keyword is used get or set followed by the normal function definition.

```
let publication = {

    text : 'some interesting text here',

    readingCounter : 0,

    changesCounter : 0,

    get content() {

        this.readingCounter++;

        return this.text;

    },

    set content(text) {

        this.text = text;

        this.changesCounter++;

    }

};

console.log(publication.content);

console.log(publication.readingCounter); // 1

publication.content = 'nothing interesting';

console.log(publication.text); // nothing interesting

console.log(publication.changesCounter); // 1
```

Methods

- A method is a function that is assigned as the property of an object. Instances of built-in object types have some built-in methods. For example every object has the `toString()` method.

```
let obj = new Object();

console.log(obj.toString()); // [object Object]
```

- To create your own method define an anonymous function and assign it to an object property.

```
let space = {

  volume : 100,

  take : function(amount) {

    let allocatedSpace = 0;

    if (this.volume >= amount) {

      this.volume = this.volume - amount;

      allocatedSpace = amount;

    } else {

      allocatedSpace = this.volume;

      this.volume = 0;

    }

    return allocatedSpace;

  }

};
```

```
space.take(10); // call the space.take() method
```

- You can add a new method to an existing object.

The this keyword

- The *this* keyword is a reference to the current object in the object's method. It is not property or variable. It cannot be inherited by a nested function. You can use it to access properties of the current object, to call its methods or to return the current object.

```
int myTest = {

    data : '###',

    printData : function() {

        console.log(this.data);

    },

    test : function() {

        this.data = 'xyz';

        this.printData();

        let that = this;

        internalTest();

        function internalTest() {

            that.data = 'abc';

            that.printData();

        }

    }

}
```

```
};

myTest.test();

// Output:

// xyz

// abc
```

Object reference

- Objects in JavaScript are always passed by reference, never by copied value.
- This means that if one variable to which the object is assigned will be assigned to the other, both will refer to exactly the same object.
- The value in the case of objects is in fact pointer to an physical address where the object is stored. Thus both variables point to the same object.
- Consequently, changes made to one of them are reflected in the other.

```
let objectA = {value : 100};

let objectB = objectA;

objectB.value = 250;

console.log(objectA.value) // 250
```

Built in Objects in JS

- JavaScript has several built-in or core language objects. These built-in objects are available regardless of window content and operate independently of whatever page your browser has loaded.
- JavaScript has several built-in objects that provide functionality for various tasks.
- These built-in objects provide a lot of useful functionality that makes JavaScript a versatile programming language.

Here are some of the most commonly used built-in objects in JavaScript:

- **Math Object:** provides mathematical functions and constants
 - **String Object:** provides methods for working with strings
 - **Number Object:** provides methods for working with numbers
 - **Array Object:** provides methods for working with arrays
 - **Date Object:** provides methods for working with dates and times
 - **RegExp Object:** provides methods for working with regular expressions
 - **Boolean Object:** provides methods for working with Boolean values
 - **Object Object:** provides methods for working with objects
 - **Function Object:** provides methods for working with functions
 - **Error Object:** provides methods for working with errors
- ❖ **Math Object:** The Math object provides mathematical functions and constants, such as Math.PI (the mathematical constant pi) and Math.abs() (the absolute value function). Other methods include Math.floor() and Math.ceil(), which round numbers down and up to the nearest integer, respectively. There are also trigonometric functions like Math.sin(), Math.cos(), and Math.tan().
- ❖ **String Object:** The String object provides methods for working with strings. Some of the most commonly used methods include String.length, which returns the length of the string, String.charAt(), which returns the character at a specific index in the string, and String.concat(), which concatenates two or more strings together. There are also methods for searching and replacing text, such as String.indexOf() and String.replace().
- ❖ **Number Object:** The Number object provides methods for working with numbers. Some of the most commonly used methods include Number.toFixed(), which formats a number with a fixed number of decimal places, and Number.toString(), which converts a number to a string. There are also methods for checking if a value is a number, such as Number.isNaN().
- ❖ **Array Object:** The Array object provides methods for working with arrays. Some of the most commonly used methods include Array.length, which returns the length of the array, Array.push(), which adds an element to the end of the array, and Array.pop(), which removes the last element from the array. There are also methods for iterating over the elements in an array, such as Array.forEach().
- ❖ **Date Object:** The Date object provides methods for working with dates and times. Some of the most commonly used methods include Date.now(), which returns the current timestamp,

and `Date.getTime()`, which returns the number of milliseconds since January 1, 1970 (also known as the Unix epoch). There are also methods for getting and setting individual components of a date and time, such as `Date.getFullYear()` and `Date.setMinutes()`.

- ❖ **RegExp Object:** The `RegExp` object provides methods for working with regular expressions. Regular expressions are patterns that can be used to search for and match text. Some of the most commonly used methods include `RegExp.test()`, which tests whether a string matches a regular expression, and `RegExp.exec()`, which returns information about the matches in a string.
- ❖ **Boolean Object:** The `Boolean` object provides methods for working with Boolean values. There are only two possible Boolean values: `true` and `false`. Some of the most commonly used methods include `Boolean.valueOf()`, which returns the Boolean value of an expression, and `Boolean.toString()`, which converts a Boolean value to a string.
- ❖ **Object Object:** The `Object` object provides methods for working with objects. Every value in JavaScript is an object, so this object provides a lot of functionality. Some of the most commonly used methods include `Object.keys()`, which returns an array of the keys in an object, and `Object.assign()`, which copies the properties of one object to another object.
- ❖ **Function Object:** The `Function` object provides methods for working with functions. Functions are the building blocks of JavaScript programming, and this object provides a lot of functionality for creating and using functions. Some of the most commonly used methods include `Function.call()`, which calls a function with a specified `this` value, and `Function.apply()`, which calls a function with a specified `this` value and arguments.
- ❖ **Error Object:** The `Error` object provides methods for working with errors. When an error occurs in JavaScript code.

Date Object in JS

- ☑ The JavaScript date object can be used to get year, month and day. You can display a timer on the webpage by the help of JavaScript date object.
- ☑ You can use different Date constructors to create date object. It provides methods to get and set day, month, year, hour, minute and seconds

Constructors

- ☑ You can use different variant of Date constructor to create date object.

`Date()`

`Date(dateString)`

`Date(year, month)`

`Date(year, month, day)`

`Date(year, month, day, hours)`

`Date(year, month, day, hours, minutes)`

`Date(year, month, day, hours, minutes, seconds)`

`Date(year, month, day, hours, minutes, seconds, ms)`

`Date(milliseconds)`

- ☑ `new Date()` creates a date object with the current date and time

```
const d = new Date();
```

- ☑ `new Date(date string)` creates a date object from a date string.

```
const d = new Date("October 13, 2014 11:13:00");
```

```
const d = new Date("2022-03-25");
```

- ☑ `new Date(year, month, ...)` creates a date object with a specified date and time.

- 7 numbers specify year, month, day, hour, minute, second, and millisecond (in that order)

```
const d = new Date(2018, 11, 24, 10, 33, 30, 0);
```

Notes:

→ JavaScript counts months from 0 to 11:

→ January = 0.

→ December = 11.

- Specifying a month higher than 11, will not result in an error but add the overflow to the next year.

```
const d = new Date(2018, 15, 24, 10, 33, 30);
```

Is the same as:

```
const d = new Date(2019, 3, 24, 10, 33, 30);
```

- Specifying a day higher than max, will not result in an error but add the overflow to the next month:

```
const d = new Date(2018, 5, 35, 10, 33, 30);
```

Is the same as:

```
const d = new Date(2018, 6, 5, 10, 33, 30);
```

- 6 numbers specify year, month, day, hour, minute, second.

```
const d = new Date(2018, 11, 24, 10, 33, 30);
```

- 5 numbers specify year, month, day, hour, and minute:

```
const d = new Date(2018, 11, 24, 10, 33);
```

- 4 numbers specify year, month, day, and hour:

```
const d = new Date(2018, 11, 24, 10);
```

- 3 numbers specify year, month, and day:

```
const d = new Date(2018, 11, 24);
```

- 2 numbers specify year and month:

```
const d = new Date(2018, 11);
```

- You cannot omit month. If you supply only one parameter it will be treated as milliseconds

```
const d = new Date(2018);
```

Notes:

- JavaScript stores dates as number of milliseconds since January 01, 1970.
- Zero time is January 01, 1970 00:00:00 UTC.
- One day (24 hours) is 86 400 000 milliseconds.

- ☑ new Date(milliseconds) creates a new date object as milliseconds plus zero time:

01 January 1970 **plus** 100 000 000 000 milliseconds is:

```
const d = new Date(100000000000);
```

January 01 1970 **minus** 100 000 000 000 milliseconds is:

```
const d = new Date(-100000000000);
```

January 01 1970 **plus** 24 hours is:

```
const d = new Date(24 * 60 * 60 * 1000);
```

```
// or
```

```
const d = new Date(86400000);
```

01 January 1970 **plus** 0 milliseconds is:

```
const d = new Date(0);
```

- ☑ When you display a date object in HTML, it is automatically converted to a string, with the toString() method
- ☑ The toDateString() method converts a date to a more readable format:

```
const d = new Date();
```

```
d.toDateString();
```

- ☑ getTime() returns the number of milliseconds since **January 1, 1970 00:00:00**.

JavaScript Date Methods

- The list of JavaScript date methods with their descriptions are listed below:

Methods	Description

getTime()	It returns the number of milliseconds since January 1, 1970 00:00:00
getDate()	It returns the integer value between 1 and 31 that represents the day for the specified date on the basis of local time.
getDay()	It returns the integer value between 0 and 6 that represents the day of the week on the basis of local time.
getFullYear()	It returns the integer value that represents the year on the basis of local time.
getHours()	It returns the integer value between 0 and 23 that represents the hours on the basis of local time.
getMilliseconds()	It returns the integer value between 0 and 999 that represents the milliseconds on the basis of local time.
getMinutes()	It returns the integer value between 0 and 59 that represents the minutes on the basis of local time.
getMonth()	It returns the integer value between 0 and 11 that represents the month on the basis of local time.
getSeconds()	It returns the integer value between 0 and 60 that represents the seconds on the basis of local time.
getUTCDate()	It returns the integer value between 1 and 31 that represents the day for the specified date on the basis of universal time.
getUTCDay()	It returns the integer value between 0 and 6 that represents the day of the week on the basis of universal time.
getUTCFullYear()	It returns the integer value that represents the year on the basis of universal time.
getUTCHours()	It returns the integer value between 0 and 23 that represents the hours on the basis of universal time.

getUTCMinutes()	It returns the integer value between 0 and 59 that represents the minutes on the basis of universal time.
getUTCMonth()	It returns the integer value between 0 and 11 that represents the month on the basis of universal time.
getUTCSeconds()	It returns the integer value between 0 and 60 that represents the seconds on the basis of universal time.
setDate()	It sets the day value for the specified date on the basis of local time.
setDay()	It sets the particular day of the week on the basis of local time.
setFullYear()	It sets the year value for the specified date on the basis of local time.
setHours()	It sets the hour value for the specified date on the basis of local time.
setMilliseconds()	It sets the millisecond value for the specified date on the basis of local time.
setMinutes()	It sets the minute value for the specified date on the basis of local time.
setMonth()	It sets the month value for the specified date on the basis of local time.
setSeconds()	It sets the second value for the specified date on the basis of local time.
setUTCDate()	It sets the day value for the specified date on the basis of universal time.
setUTCDay()	It sets the particular day of the week on the basis of universal time.
setUTCFullYear()	It sets the year value for the specified date on the basis of universal time.
setUTCHours()	It sets the hour value for the specified date on the basis of universal time.

setUTCMilliseconds()	It sets the millisecond value for the specified date on the basis of universal time.
setUTCMinutes()	It sets the minute value for the specified date on the basis of universal time.
setUTCMonth()	It sets the month value for the specified date on the basis of universal time.
setUTCSeconds()	It sets the second value for the specified date on the basis of universal time.
toDateString()	It returns the date portion of a Date object.
toISOString()	It returns the date in the form ISO format string.
toJSON()	It returns a string representing the Date object. It also serializes the Date object during JSON serialization.
toString()	It returns the date in the form of string.
toTimeString()	It returns the time portion of a Date object.
toUTCString()	It converts the specified date in the form of string using UTC time zone.
valueOf()	It returns the primitive value of a Date object.

Example:

```

html>

<head>

    <title>Get date and time in JavaScript.</title>

</head>

<body>

    <h2>Get date and time using JavaScript.</h2>

```

```
<h4>Output after using the basic methods of the Date()  
class.</h4>
```

```
<div id="date"> </div>
```

```
<div id="time"> </div>
```

```
<script type="text/javascript">
```

```
    let date = document.getElementById("date");
```

```
    let time = document.getElementById("time");
```

```
    // creating the date object and getting the date and time
```

```
    let newDate = new Date();
```

```
    let year = newDate.getFullYear();
```

```
    let month = newDate.getMonth();
```

```
    let todaySDate = newDate.getDate();
```

```
    let hours = newDate.getHours();
```

```
    let minutes = newDate.getMinutes();
```

```
    let seconds = newDate.getSeconds();
```

```
    date.innerHTML = " " + todaySDate + "/" + month + "/" +  
year;
```

```
    time.innerHTML = hours + ":" + minutes + ":" + seconds;
```

```
</script>
```

```
</body>
```

```
</html>
```

Tasks

Write a Script to calculate the age of a user by taking date of birth (through prompt or text field)

Write a Script to display digital clock in a web page.

Interacting with browsers

- The Browser Object Model (BOM) is used to interact with the browser.
- The Browser Object Model (BOM) allows JavaScript to "talk to" the browser.
- The BOM provides you with objects that expose the web browser's functionality.
- The BOM provides a way for JavaScript code to interact with the browser and manipulate its behavior, allowing developers to create dynamic and interactive web applications.
- The BOM includes the properties and methods for JavaScript to interact with the web browser.
- JavaScript provides different objects like *window*, *navigator*, *history*, *location*, *screen*, etc to interact with browsers.

window Object

- The global object of JavaScript in the web browser is the **window** object. It means that all variables and functions declared globally with the *var* keyword become the properties and methods of the **window** object.
- It represents the current browser window and provides methods for manipulating the window, such as opening or closing it, changing its size or position, and navigating to a new URL.

- The window object is supported by all browsers. It represents the browser's window.
- All global JavaScript objects, functions, and variables automatically become members of the window object.
- Global variables are properties of the window object.
- Global functions are methods of the window object.
- Even the document object (of the HTML DOM) is a property of the window object.
- The window object in JavaScript represents the current browser window or tab and is the top-level object of the Browser Object Model (BOM). It provides a global namespace for all JavaScript code running within the context of the window.

Window size

- ☑ The window object has four properties related to the size (in pixels) of the window:
 - **window.innerHeight:** This property represents the height of the browser window's viewport, including any scrollbars, but excluding the toolbar and other browser elements.
 - **window.innerWidth:** This property represents the width of the browser window's viewport, including any scrollbars, but excluding the toolbar and other browser elements.
 - **window.outerHeight:** This property represents the height of the entire browser window, including any toolbar and other browser elements.
 - **window.outerWidth:** This property represents the width of the entire browser window, including any toolbar and other browser elements.

```
let pageHeight = window.innerHeight;  
let windowHeight = window.outerHeight
```

[NOTE: All these properties are read-only.]

- ☑ The window object provides two methods which you can use to resize a window.
 - The **resizeTo()** expects two arguments: a new width and height and resizes window to given dimensions.

- The **resizeBy()** in turn expects two arguments which are differences in each size.

```
window.resizeTo(600, 400); // resize to 600 x 400
window.resizeBy(200, 100); // resize to 800 x 500
```

Window position

- ☑ The position of a window specify two properties: The **window.screenX** and **window.screenY** indicate coordinates relative to the screen in CSS pixels.
- ☑ This is a distance of the left and top border of the browser window from the left and upper point of the screen. Both properties are read-only.

```
let left = window.screenX;
let top = window.screenY;
```

- ☑ To move the window to a new position the window object provides two methods.
 1. The **window.moveTo()** expects the x and y coordinates to move to
 2. The **window.moveBy()** also expects two arguments, number of pixels to move in each direction.

```
// move the window to the position x = 10, y = 20
window.moveTo(10, 20);

// move the window right by 50 pixels and down by 30 pixels
window.moveBy(50, 30);
```

Opening and closing windows

- ☑ JavaScript allows you to open new browser windows (or tabs, depending on a browser configuration) from the current window.
- ☑ The **window.open()** method opens specified URL in a new window (or in existing one). It accepts following optional arguments:
 1. The **url** specifies the URL of the document to load. If it is not supplied, a blank page is displayed and the URL is set to about:blank.

```
window.open('http://example.com');
```

2. The second argument, **target** specifies the name of a window. If a window with this name already exists, that window is used to load the URL. Otherwise a new window with the given name is created. If this argument is omitted the special window name `_blank` is used and it opens an unnamed window.

```
window.open('http://example.com', 'myWindow');
```

There are four special window names that may be used:

- ☞ **_blank** to open a new blank window,
- ☞ **_self** to load a new URL into the current window,
- ☞ **_parent** to use parent window and
- ☞ **_top** to use top level ancestor window.

3. The third argument is the **settings**. It is comma-separated list of window settings (without spaces). If it is omitted a new browser window is opened with the default settings. See below the list of possible settings:

Setting	Possible value	Description
left	Number	The left position of the new window
top	Number	The top position of the new window
width	Number	The width of the new window, minimum value is 100
height	Number	The height of the new window, minimum value is 100
menubar	["yes", "no", 1, 0]	Specifies whether to display the menu bar, the default is "no"
location	["yes", "no", 1, 0]	Specifies whether to display (enable) the location bar, the default depends on the browser
toolbar	["yes", "no", 1, 0]	Specifies whether to display the status bar, the default is "no"
status	["yes", "no", 1, 0]	Specifies whether to display the status bar, the default depends on browser
scrollbars	["yes", "no", 1, 0]	Specifies whether scrolling is allowed, the default is "no"
resizable	["yes", "no", 1, 0]	Specifies whether resizing is allowed, the default is "no"

```
window.open("https://example.com",_blank,"width=300,height=250,toolbar=yes,status=yes");
```

4. The fourth argument **replace** specifies whether the URL should replace the current entry in the history of browsing or should create a new entry. The following values are supported:

true - URL replaces the current document in the history list

false - URL creates a new entry in the history list

- ☑ The **window.open()** methods returns a reference to the new window. It is the Window object and you can use it to refer to this new window.
- ☑ To close a window simply use **window.close()** method.

```
window.close();
```

Note that browsers only allow you to automatically close the windows created by **window.open()** of the same origin. Closing the main window is not possible automatically.

Some methods in window object

The important methods of window object are as follows:

Method	Description
alert()	displays the alert box containing message with ok button.
confirm()	displays the confirm dialog box containing message with ok and cancel button.
prompt()	displays a dialog box to get input from the user.
open()	opens the new window.
close()	closes the current window.
setTimeout()	performs action after specified time like calling function, evaluating expressions etc.
resizeTo()	resizes the current window
moveTo()	moves the current window to a specified coordinate

Example of window.alert()

It displays alert dialog box. It has message and ok button

```
<script type="text/javascript">
function msg(){
    window.alert ("Hello Alert Box");
}
</script>
<input type="button" value="Click" onclick="msg()"/>
```

Example of window.confirm()

It displays the confirm dialog box. It has message with ok and cancel buttons.

```
<script type="text/javascript">
    function msg(){
        let v= confirm("Are you sure?");
        if(v==true){
            window.alert ("OK");
        }
        else{
            window.alert ("CANCEL");
        }
    }
</script>

<input type="button" value="Delete" onclick="msg()"/>
```

Example of window.prompt()

It displays prompt dialog box for input. It has message and textfield.

```
<script type="text/javascript">
    function msg(){
        let v= window.prompt ("Who are you?");
        window.alert("You are "+v);
    }
</script>

<input type="button" value="Click" onclick="msg()"/>
```

Example of window.open()

It displays the content in a new window.

```
<script type="text/javascript">
    function msg(){
        window.open("http://www.cdcsit.edu.np");
    }
</script>
<input type="button" value="CDCSIT" onclick="msg()"/>
```

Dialog boxes

- Browser window allows you to display simple dialog boxes to the user.
- There are **alert()**, **confirm()** and **prompt()** methods.
- These dialogs do not contain HTML and CSS formatting but plain text. Their appearance depend on browser and operating system.
- In addition, all these dialogs are blocking. When the dialog is displayed a further code execution stops until the user dismisses this dialog. So if the dialog is displayed when the page is loaded, further rendering is stopped until the user closes this dialog.
- The **alert()** method displays a message to the user.

- ✎ It accepts a string as an argument. When the **alert()** is called, the browser displays dialog box with passed text and below a OK button.
- ✎ The only possibility for the user is to dismiss the dialog after displaying the message.

```
alert("welcome to the website");
```

- The **confirm()** method displays a message to the user and expects confirmation.
 - ✎ The method accepts a string as an argument.
 - ✎ Dialog box in addition to the text message displays two buttons: **OK** and **Cancel**.
 - ✎ The user can confirm or cancel the message. If the user clicks **OK**, the **confirm()** returns true, if he clicks **Cancel** it returns false. This allows the user to decide on the next action.

```
let confirmed = confirm("Are you sure to delete this item?");
if (confirmed) {
    // here the code that removes the item
}
```

- The **prompt()** method displays a message and prompts the user to enter a data.
 - ✎ The method accepts two string arguments: the text to display and the default value for the input.
 - ✎ Dialog box displays a text message, text input, where user can enter a data and buttons **OK** and **Cancel**. It returns entered data if user clicks OK and null otherwise.

```
let favorite = prompt("Your favorite programming language:", "JavaScript");
```

Timers

- JavaScript allows you to run the code at specific time points and execute it once or repeatedly. For this you can use the global functions **setTimeout()** and **setInterval()** which are in fact methods of the Window.
- The **setTimeout()** method allows you to execute code after a specific amount of time. It accepts two arguments: the code to execute and the number of milliseconds to wait before

executing the code. After calling the **setTimeout()** returns an unique identifier of the timeout.

```
let displayTimMsg = function() {
    let msg = "Time is up!";
    alert(msg);
};
setTimeout(displayTimMsg, 2000);
```

- The **setInterval()** method is similar to the **setTimeout()**. The difference between them is that the **setInterval()** executes code repeatedly at intervals of a specific time. This is done until the page is unloaded or the interval canceled. The **setInterval()** also accepts two arguments: the code to execute and time in milliseconds to wait between executions. After calling the **setInterval()** returns an unique identifier of the interval.

```
var displayIntMsg = function() {
    var msg = "The next interval has passed!";
    alert(msg);
};
setInterval(displayIntMsg, 3000);
```

- The **clearInterval()** method clears a timer set with the **setInterval()** method.

```
myInterval = setInterval(function, milliseconds);

clearInterval(myInterval);
```

- Note that both **setTimeout()** and **setInterval()** accept as the code to execute function or string. However the second way is not recommended.

```
// Not recommended:
setTimeout("let msg = 'Hello!'; alert(msg);", 2000);

// Recommended:
setTimeout(function(){
    let msg = 'Hello!';
    alert(msg);
}, 2000);
```

The location object

- The **location** is the property of the Window object type and refers to object of the Location object type. It represents the URL of the currently loaded document in the browser window. The properties of the location object provide detailed information about the url and its parts:
- The href property is a string that contains full URL of the currently loaded page. For example The **toString()** method returns the value of this property. So in the context that will call **toString()** you can simply write location instead of **location.href** to get the full url.
- The **protocol** property contains the protocol used by the page (usually http: or https:).
- The **hostname** property returns the name of the server (usually domain).
- The **host** property returns the name of the server and the port (if presented).
- The **port** property returns the port of the request if specified or empty string otherwise.
- The **pathname** property returns the path identified the specific resource (directory, filename).
- The **search** property returns the query string, beginning with a question mark (?) list of key-value pairs of params separated by ampersand (&) and specifying details of the request.
- The **hash** property returns the optional fragment identifier, it begins with hash mark (#) and is followed by an element ID in the document.

```
//assume that current URL is:
```

```
// http://www.example.net/api/?type=course&langauge=js#topics
```

```
// location properties values:
```

```
location.href; //http://www.example.net/api/?type=course&langauge=js#topics
```

```
location.protocol; // http:
```

```
location.hostname; // www.example.net

location.host; // www.example.net:8080

location.port; // 8080

location.pathname; // /api/

location.search; // ?type=course&langauge=js

location.hash; // #topics
```

Changing the location

The location can be changed is several ways. You can simply assign a new URL to the location href property:

```
location.href = 'http://example.net';
```

Even simpler, you can assign a new URL directly to the location:

```
location = 'http://example.net';
```

The next option is to use the assign() method:

```
location.assign('http://example.net');
```

Not only the entire URL but also its individual parts can be changed by changing the properties of the location object.

```
// Assume that there is loaded a page with the URL

// http://www.example.net/api/?type=course#topics

location.hostname = 'www.example.com'; // changes URL to

//http://www.example.com/api/?type=course#topics

location.pathname = 'api2'; // changes URL to
```

```
//http://www.example.net/api2/?type=course#topics
```

```
location.search = '?type=course&language=js'; // changes URL to
```

```
// http://www.example.net/api/?type=course&language=js#topics
```

```
location.hash = '#resources'; // changes URL to
```

```
// http://www.example.net/api/?type=course#resources
```

Change of a location property, except of the hash, causes that the page reloads.

Location Object Methods

Method	Description
<code>assign()</code>	Loads a new document
<code>reload()</code>	Reloads the current document
<code>replace()</code>	Replaces the current document with a new one

- **location.assign(url):** Loads the specified URL in the current window.
location.assign("https://www.example.com/page2.html");
- **location.replace(url):** Loads the specified URL in the current window, but replaces the current page in the browser history.
location.replace("https://www.example.com/page2.html");
- **location.reload():** Reloads the current page.
location.reload();

The navigator object

- The navigator is the property of the Window object type and refers to object of the Navigator object type. It provides information about the user's browser, such as the vendor and the version of the browser.
- The navigator contains number of properties which describe client-side data. See the most important of them below:
 - The **appName** is the full name of the browser. But for historical reasons (in order to display the content correctly) most browsers have set it to "Navigator"
 - The **appCodeName** – the name of the browser, not very useful, as most of browsers have it set to "Mozilla".
 - The **appVersion** – browser version, the number 5.0 at the beginning indicates fifth-generation browser, then information on browser vendor and version, it may not be accurate.
 - The **cookieEnabled** – indicates whether cookies are enabled.
 - The **geolocation** – object providing information on location of the user.
 - The **language** – returns primary language of the browser.
 - The **mimeTypes** – returns mime types supported by the browser.
 - The **onLine** – indicates whether the browser is online.
 - The **oscpu** – information about operating system.
 - The **platform** – information about system platform, it may not be correct.
 - The **plugins** – list of plugins installed in the browser.
 - The **product** – always is set to "Gecko", it is kept from historical reasons.
 - The **userAgent** – string providing information about the browser name, version and operating system, it is used to identify user's browser.
 - The **activeVRDisplays** – returns list of VRDisplay objects.
 - The **connection** – returns object providing information on the network connection of a device.
 - The **serviceWorker** – returns ServiceWorkerContainer object, which provides access to ServiceWorker objects.

The history object

- The **history** is the property of the Window object type and refers to the object of the History object type.
- The history object represents the browsing history of a window. However for security reasons it is not possible to get the URLs which user has visited. Instead of this, the history object provides methods for navigating back and forward in the browsing history without knowing the urls of visited pages:

- The **back()** method works like the browser's Back button. It causes that the browser go backward one step in the browsing history and displays the previous page.

```
history.back(); // go back one step
```

- The **forward()** method works like the browser's Forward button. It causes that the browser go forward one step in the browsing history and displays the following page.

```
history.back(); // go forward one step
```

- The **go()** method navigates in both directions of browsing history, backward and forward, additionally it can skip any number of pages. The **go()** method accepts an integer argument, that indicates the number of pages to go back (for negative numbers), or to go forward (for positive numbers);

```
history.go(-2); // go back two pages
```

```
history.go(1); // go forward one page
```

- The history object provides also the the length property. It specifies the number of items in the browsing history list.

Frames

- Frames are older HTML feature that allowed web developers to split a web page into multiple independent sub-windows, each with its own HTML document and content.
- JavaScript has some functionality that can interact with frames, but frames themselves are largely considered **deprecated** and **not recommended for modern web development**.

- Each frame is defined using an HTML **frame** or **iframe** tag, which specifies the URL of the content to be displayed in the frame.
 - For example, `<frame src="subpage.html">` would create a frame with the contents of `subpage.html`.
- Each frame has its own window object and global JavaScript context. JavaScript code running in one frame cannot directly access or modify the variables or functions of another frame.
- However, JavaScript code in one frame can interact with another frame by accessing its window object.
 - For example, `window.frames[0]` would reference the first frame in the current document, and you could then call methods on that object to manipulate its content.
- **window.frames** represents a list of frame objects. It is similar to an array in that it has a **length** property and its items can be accessed using the **[i]** notation.

```
const frames = window.frames; // or const frames = window.parent.frames;
for (let i = 0; i < frames.length; i++) {
  // do something with each subframe as frames[i]
  frames[i].document.body.style.background = "red";
}
```

- Cross-frame scripting is a security risk, so most modern web browsers enforce a same-origin policy that prevents JavaScript code from accessing or modifying content in frames that have a different origin (i.e. a different domain, port, or protocol).
- In modern web development, frames have largely been replaced by other techniques such as AJAX, server-side includes, and client-side JavaScript libraries like React and Angular. While frames may still be used in some cases, they are generally not recommended due to their limitations and security risks.

Document Object Model (DOM)

The DOM is a W3C (World Wide Web Consortium) standard.

The DOM defines a standard for accessing documents:

"The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document."

The W3C DOM standard is separated into 3 different parts:

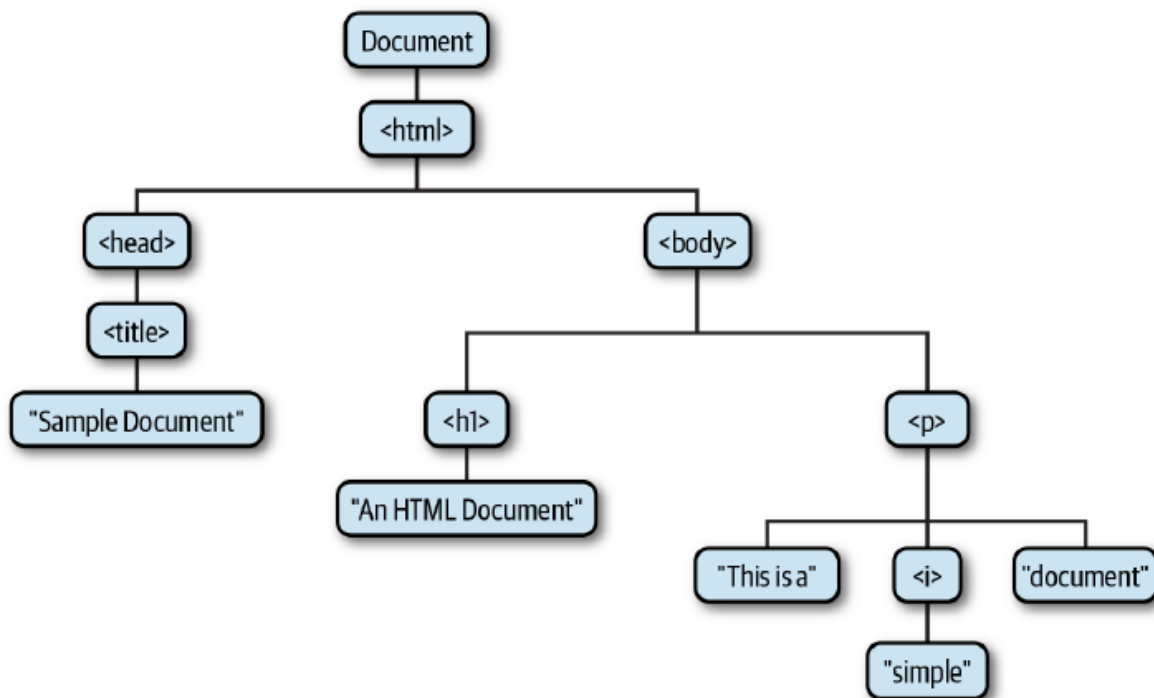
- Core DOM - standard model for all document types
- XML DOM - standard model for XML documents
- HTML DOM - standard model for HTML documents
- One of the most important objects in client-side JavaScript programming is the Document object—which represents the HTML document that is displayed in a browser window or tab.
- The Window object has a **document** property, which is reference to a Document object and is also available as a global variable. The Document object represents the content of the window and is the main object of Document Object Model.
- Document Object Model (DOM) is an application programming interface (API) for HTML documents.
- The DOM represents a HTML **document as a tree structure of objects**.
 - ☞ Each node represents an element in the document.
 - ☞ Elements of the document can be added, removed and modified.

Document Structure

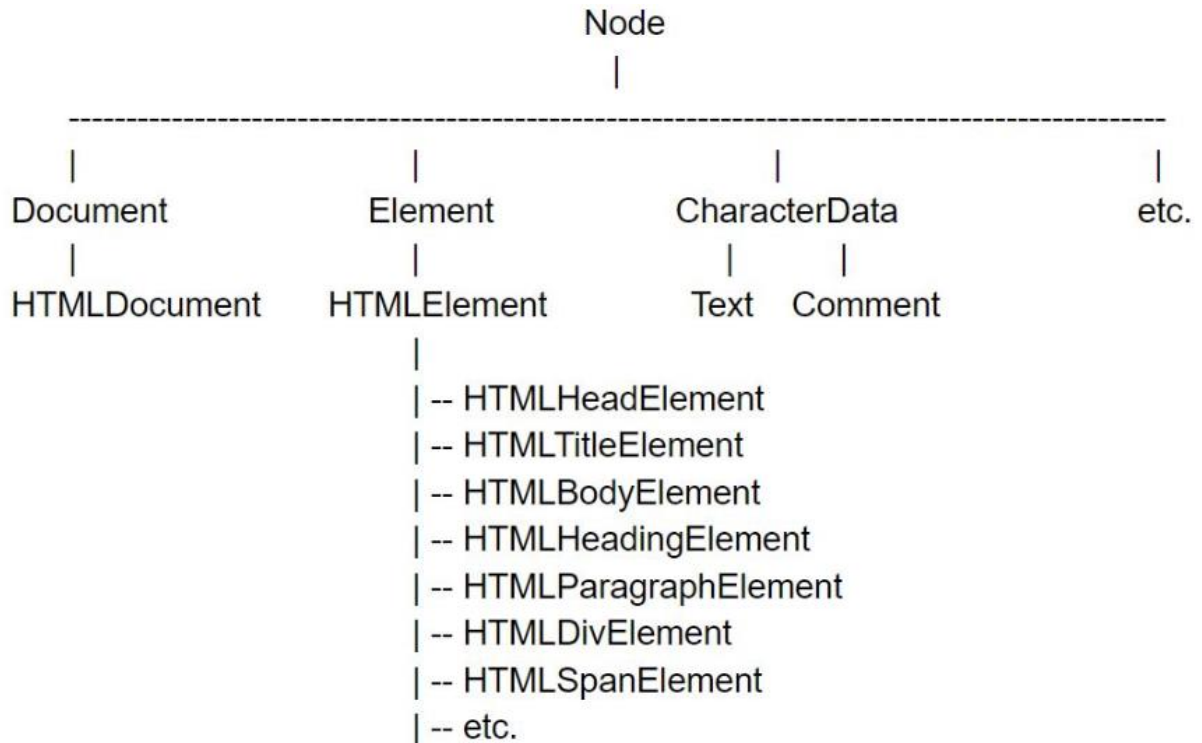
- ☒ HTML document can be treated as a tree of nodes. Consider the following HTML document:
Consider the following simple HTML document:

```
<html>
  <head>
    <title>Sample Document</title>
  </head>
  <body>
    <h1>An HTML Document</h1>
    <p>This is a <i>simple</i> document.
  </body>
</html>
```

- ☑ The DOM API mirrors the tree structure of an HTML document.
 - For each HTML tag in the document, there is a corresponding JavaScript Element object, and for each run of text in the document, there is a corresponding Text object.
 - The Element and Text classes, as well as the Document class itself, are all subclasses of the more general Node class, and Node objects are organized into a tree structure that JavaScript can query and traverse using the DOM API.
- ☑ The DOM representation of above html document is the tree pictured below:



- ☑ Each HTML tag is treated as a node of the tree. Each node in the tree is represented by a Node object. The root is the Document node, that represents the entire document. HTML elements (like <head>, <body>, <div>, etc) are represented by Element nodes, texts are represented by Text nodes, comments by Comment nodes, etc. The Node is the most generic type. Its subtypes represent specific types of HTML elements. Figure below presents hierarchy tree of nodes:



Nodes and Tree Traversals

- ❖ Structure of the HTML document is like tree of Node objects. The Node object type defines properties that contain information about the node and the relationship between nodes.
- ❖ The **nodeType** property specifies the node type of HTML element. Its value can be a number from 1 to 12.
- ❖ Below the list of current node types and their values:

```

Node.ELEMENT_NODE (1)
Node.TEXT_NODE (3)
Node.PROCESSING_INSTRUCTION_NODE (7)
Node.COMMENT_NODE (8)
Node.DOCUMENT_NODE (9)
Node.DOCUMENT_TYPE_NODE (10)
Node.DOCUMENT_FRAGMENT_NODE (11)
  
```

- ❖ The **nodeValue** property holds the content of Text nodes.
- ❖ The **nodeName** property holds tag name of an element.
- ❖ The properties that allow to traverse tree of nodes are as follows:
 - ☞ The **parentNode** specifies parent node of current one.
 - ☞ The **childNodes** holds the list of child nodes as a **NodeList** object.
 - ☞ The **firstChild** specifies the first child of a node.
 - ☞ The **lastChild** specifies the last child of a node.
 - ☞ The **nextSibling** specifies the next sibling of a node. Siblings are nodes with the same parent.
 - ☞ The **previousSibling** specifies the previous sibling.
 - ☞ Assuming that **myNode** is a node in the tree representing a selected element you can find other associated nodes and traverse the tree:

```
// get parent node of myNode
myNode.parentNode;

// get first child of myNode
myNode.firstChild;

//get second child of myNode
myNode.childNodes[1];

//get next sibling of myNode
myNode.nextSibling;

// get second sibling of myNode
if (myNode.nextSibling) {
    myNode.nextSibling.nextSibling;
}
```

Selecting Document Elements

- ☞ Elements of the document can be read and modified. In order to do this, first you have to select the element. The DOM provides several ways to obtain an element.

Selecting element by ID

- ☞ Every HTML element can have an id attribute, which should be unique in the document.

```
<div id="quantity">10</div>
```

- ☞ You can select an element using its unique ID with the **getElementById()** method of the Document object type. This method returns Element object.

```
let quantityElement = document.getElementById("quantity");
```

Selecting elements by name

- ☞ HTML elements can have a name attribute, usually used by form elements. The value of a name attribute do not have to be unique.

```
<input type="radio" name="pet" value="cat"> Cat<br>
<input type="radio" name="pet" value="dog"> Dog
```

- ☞ You can select all elements with specific name using the **getElementByName()** method of the **HTMLDocument** object type. This method returns **NodeList** object, which is like a read-only array of **Element** objects. It is available only for HTML documents (not XML documents).

```
let petsList = document.getElementsByName('pet');
```

- ❖ Setting the name attribute for some HTML elements like <form>, , <object>, <embed>, <iframe> automatically creates properties with those names in the Document object. If there is only one element with a specific name, the value of a related property is this element. If there is more elements, then the value of the property is NodeList object.

```
<form name="contactForm">
```

```
let contactForm = document.contactForm;
```

Selecting elements by tag type

- ❖ HTML elements can be selected by their tag type using **getElementsByTagName()** method of the **Document** object type. This method returns a **NodeList** object containing found elements in the order in which they appear in the document.

```
 <br>

```

```
let imagesList = document.getElementsByTagName('img');
```

- ❖ For certain types of HTML elements like <form>, , <a> **HTMLDocument** defines properties (forms, images, links) which allow easy access to these elements. These properties return **HTMLCollection** object which is similar to **NodeList**.

```
var imagesList = document.images;
```

Important thing about **NodeList** and **HTMLCollection** is that returned objects are dynamic. It means that they update **NodeList** or **HTMLCollection** of elements when the document changes.

Selecting elements by CSS class

- ❖ HTML elements can have class attribute through which you can specify CSS classes, which define presentation styles. HTML elements store value of class attribute in the **className** property. To select all elements with a given class or classes use **getElementsByClassName()** method and specify required classes separated by space. It returns **NodeList** object.

```
<div class="content">Content 1</div>
<p class="content">Content 2</p>
```

```
let contentList = document.getElementsByClassName('content')
```

Selectors API

- In addition to previously discussed methods of selecting elements W3C defines The Selectors API, which allows you to find elements that match a specific CSS selector.
- The most important are two methods: *querySelectorAll()* and *querySelector()*. The difference between them is that the former returns all matched elements and the latter returns only the first matched element. Both these methods are available on the Document, the Element and the DocumentFragment object types.
- The *querySelectorAll()* method accepts CSS query as an argument and returns NodeList object with all matched elements.

Unlike previously discussed methods, returned NodeList objects by Selectors API methods are static. It means that they do not update when the document changes

```
// select all elements in the document with CSS class "item"
let itemList = document.querySelectorAll('.item');

// select all paragraph elements in the document with CSS class "note"
let notesList = document.querySelectorAll('p.note');

// inside an element with a "content" id select all elements with CSS
//class "post"
document.getElementById('content').querySelectorAll('.post')
```

The *querySelector()* method also accepts CSS query as an argument and returns the first matched element.

```
// inside the element with a "menu" id select the first link
//with CSS class "internal" nested in the span tag
document.getElementById('menu').querySelector('span a.internal')
```

Element Content

- Element content can be considered as HTML, plain text or Text Nodes.

HTML content

- The Element has the *innerHTML* property which holds the content of that element. You can use the *innerHTML* property to read content of an element as well as to write a new content.

```
<p id="note">A small <b>portion</b> of the content.</p>
```

```
// returns: "A small <b>portion</b> of the content."
var htmlContent = document.getElementById('note').innerHTML;
```

```
// result: "<p id='note'>A new <i>portion</i> of the
content!</p>"
document.getElementById('note').innerHTML = 'A new
<i>portion</i> of the content!';
```

The element has also the *outerHTML* property. It holds the content of the element together with the surrounding opening and closing tags of that element. When you assign new content to *outerHTML* property, you replace the element itself.

```
<p id="note">A small <b>portion</b> of the content.</p>
```

```
// returns "<p id='note'>A small <b>portion</b> of the
content.</p>"
var htmlContent = document.getElementById('note').outerHTML;

// result: "<div>A new <i>portion</i> of the content!</div>"
document.getElementById('note').outerHTML = '<div>A new
<i>portion</i> of the content!</div>';
```

Plain text content

- ☑ Another option is to read or write element's content as a plain text. You can do this using the `textContent` property of an Element.

```
<p id="note">A small <b>portion</b> of the content.</p>
```

```
// returns: "A small portion of the content."
document.getElementById('note').textContent;

// assigns: "A new content."
document.getElementById('note').textContent = "A new content.";
```

Text Nodes content

- ☑ Third possibility to deal with the content of an element is to use its child text nodes. In text nodes the content is holded in the *nodeValue* property.

```
<p id="note">A simple content.</p>
```

```
let content = '';
let childNode = document.getElementById('note').firstChild;
if (childNode != null && childNode.nodeType == Node.TEXT_NODE) {
    content = childNode.nodeValue;
}
console.log(content); // A simple content.
```

Creating and manipulating Nodes

- ☑ In addition to changing the content of elements, it is also possible to modify the document at the node level.

Creating Nodes

- ☑ To create a new Element node you can use the ***createElement()*** method of the Document object type. It accepts the name of the tag element as an argument.

```
let elementNode = document.createElement('p');
```

Similarly, to create a new Text node you can use the ***createTextNode()*** method. It accepts a string content as an argument.

```
let textNode = document.createTextNode('Text node content');
```

Inserting Nodes

- A newly created node is not automatically added into the document. You can use ***appendChild()*** or ***insertBefore()*** methods of the Node object type to insert a node into the document.
- The ***appendChild()*** inserts a node passed as an argument as the last child of the element on which it is invoked.

```
<div id = 'div1'></div>
```

```
let myNewHeading = document.createElement('h1');

let myTextNode = document.createTextNode("This is demo .....");

myNewHeading.appendChild(myTextNode);

let myDiv = document.getElementById('div1');

myDiv.appendChild(myNewHeading);
```

- The ***insertBefore()*** method is similar to the ***appendChild()***. It also inserts a node passed as the first argument as a child of the element on which it is invoked. However the second argument allows you to specify before which child node given node is to be inserted. If the second argument is null, the node is inserted as a last child.

```
let nextHeading = document.createElement('h4');

let nextTextNode = document.createTextNode("Hello Hello...");

nextHeading.appendChild(nextTextNode);

myDiv.insertBefore(nextHeading,myNewHeading);
```

- The ***appendChild()*** and the ***insertBefore()*** can be used not only to insert new nodes but also to change location of a node in the document. If you call one of these methods to insert a node

that is already in the document, it will be removed from previous place and insert to a new one.

Replacing Nodes

- It is possible to replace one node by an another. The **replaceChild()** method replaces a child node by a new node. A new node is accepted as the first argument and a node to be replaced as the second argument. This method is invoked on the parent node of the node to be replaced.

```
<h3><div id="container"><span>Text 1</span><span>Text 2</span></div></h3>
```

```
let divNode = document.getElementById('container');  
let lastChild = divNode.lastChild;  
let textNode = document.createTextNode('Alternative text');  
divNode.replaceChild(textNode, lastChild);
```

Removing Nodes

To simply remove a node use **removeChild()** method. It accepts a node to be removed as the argument and it is invoked on the parent node of the node to be removed.

```
myDiv.removeChild(myDiv.firstChild);
```

Attributes

- HTML elements in addition to tag name can have attributes that are name-value pairs. You can read and write values of these attributes thru properties of HTML element object type.
- HTML element object type that represents elements in the HTML document specifies properties that reflect the standard attributes of HTML elements. It includes general purpose attributes (such as id, title), element-specific attributes (src, href, action), event-handler attributes (onclick).
- Property names are case sensitive. Some HTML attribute names are reserved words in JavaScript. That is why properties that represent these attributes have amended name (className for class attribute, htmlFor for for attribute)

```

```

```
<a href="http://www.example.net" class="myLink">www.example.net</a>
```

```
let image = document.images[0];
// set element id to myImg
image.id = 'myImg';
// replace image1.jpg to image2.jpg
image.src = 'image2.jpg';
```

```
let link = document.links[0]
// read URL of the link
let url = link.href;
// read CSS class of the link
let cssClass = link.className;
```

- Values stored in properties are the same type as mirrored attribute (usually string, but also may be number or boolean).
- The Element object type defines methods **getAttribute()** and **setAttribute()** that allow you to read and set HTML attributes. These methods also allow you to deal with non-standard HTML attributes. The **getAttribute()** method accepts as the argument name of the attribute and it returns always a value of string type. The **setAttribute()** method accepts as the first argument name of attribute and as the second argument a value to be set. Attributes names are case insensitive

```

```

```
// read height attribute
var height = document.getElementById('myImg').getAttribute('HEIGHT');
// set CSS class smallImg
document.getElementById('myImg').setAttribute('class', 'smallImg');
```

- The **removeAttribute()** method of Element object type allows you to delete a given attribute from the element. Using the **hasAttribute()** method you can check if an attribute exists in a given element

```
<input type="text" value="" disabled="disabled" id="myField">
```

```
document.getElementById('myField').removeAttribute('disabled');
```

```
// it returns false:
```

```
document.getElementById('myField').hasAttribute('disabled');
```

In order to HTML document was valid, non-standard HTML attributes has to begin with “data-” prefix. For example instead of attribute name “category” should be “data-category”. The Element object type defines a dataset property that holds values of attributes with “data-” prefix.

```
<div id="myDiv" data-category="training">Content...</div>
```

```
let category = document.getElementById('myDiv').dataset.category; // training
```

Writing in the document

- Document object type provides the **write()** method that allows you to write in the document. There is also **writeln()** method which is like the **write()** except that it appends a new line character (n) at the end of the output string. These methods can be used to dynamically insert content when page is being loaded
- HTML code added thru the **write()** method is treated the same as if it was included directly in the document and can be accessed through DOM API.
- The **write()** method can be used to output the content into the document only while the document is being loaded. If the **write()** method is used after the document has been loaded, it erases and overwrites the current document
- The **document.write()** method is one of the oldest API's in JavaScript and currently it is rarely used. There are better solutions to add content to the document like the **innerHTML** property.

Event Handling

- Event is simply an action performed by user or browser itself.
- Whenever something happens to the HTML document or its element, the web browser generates an event. For example loading document, mouse click, key press and element focus web browser turns into events.
- It is possible to write a JavaScript code that will be invoked when a specific event occurs.

- Event handling in JavaScript is the process of responding to user actions or system events, such as mouse clicks, key presses, and page loading.
- There are many types of events that can be handled in JavaScript, including click, mouseover, keydown, load, and many more. By handling events, you can make your web pages more interactive and responsive to user actions.
- Some important concepts related to event handling in JavaScript:
 - **event handler or event listener**
 - **event target**
 - **event type**
 - **event object**
 - **event propagation**

Event Handlers

- ☞ Event handler, called also event listener, is a function that handles an event.
- ☞ Such a function should be registered for specific event type on specific DOM element.
- ☞ Whenever that even occurs, corresponding handler is called.
- ☞ An event handler is a function that is called when an event occurs. The function is attached to an element using the `addEventListener()` method, and it is executed when the event is triggered.

Event Type

- An event type is a string which specifies the kind of event that has occurred.
 - The type “mousemove,” for example, means that the user moved the mouse.
 - The type “keydown” means that the user pressed a key on the keyboard down. And the type “load” means that a document (or some other resource) has finished loading from the network.
- Because the type of an event is just a string, it's sometimes called an **event name**, and indeed, we use this name to identify the kind of event.

Event Target

- The event target is the object on which the event occurred or with which the event is associated.
- When we speak of an event, we must specify both the type and the target. A load event on a Window, for example, or a click event on a <button> Element.
- Window, Document, and Element objects are the most common event targets in client-side JavaScript applications, but some events are triggered on other kinds of objects.
- The event target is the DOM element that the event was fired on, such as a button or a text input. You can select the target element using the *event.target* property.

Event Object

- The **event** object is a JavaScript object that contains information about the event that occurred, such as the target element, the type of event, and any additional information that may be relevant. You can access the event object using the event parameter that is passed to the event handler function.
- Event objects are passed as an argument to the event handler function.
- All event objects have a **type** property that specifies the **event type** and a **target** property that specifies the **event target**.
- Each event type defines a set of properties for its associated event object.
 - The object associated with a mouse event includes the coordinates of the mouse pointer, for example, and the object associated with a keyboard event contains details about the key that was pressed and the modifier keys that were held down.

Event Propagation

- Event propagation is the process by which events are propagated from the target element to its parent elements in the DOM tree.
- There are two types of event propagation: bubbling and capturing.
 - Bubbling is the default behavior, where the event is first handled on the target element, then on its parent elements, up to the root of the DOM tree.
 - Capturing, on the other hand, is when the event is first handled on the root element, then on its children elements, down to the target element

Registering event handlers

☞ There are several ways to register event handlers.

Registering event handler using HTML element attribute

☑ In HTML code we can set the attribute with the name of the event type on the HTML element, that is the event target.

☞ Attribute name consists of the word “on” followed by the event name (eg. onclick, onchange, onsubmit).

☞ Attribute value should be JavaScript code surrounded by quotes. That code becomes the body of the event handler function.

```
<button type="button" onclick="alert(new Date());">
```

```
  Show current date </button>
```

JavaScript code is often several lines long. It is more common to see event attributes calling functions.

```
<button onclick="displayDate()">Get Date </button>
```

☞ Some types of events like **onload** refer to the entire document, then you should put the attribute in the body element.

Registering event handler by setting object property with event type

☑ In JavaScript code you can set the property with the name of the event type of the window, document or document element, that is the event target.

- Property name, consists of the word “on” followed by the event name and should be written in lowercase.
- Property value should be a valid JavaScript function.

```
<button type="button" id="myButton">Show current date</button>
```

```
let myButton = document.getElementById("myButton");
myButton.onclick = function () {
    alert(new Date());
};
```

Registering event handler using addEventListener() method

☑ We can use a standard method ***addEventListener()***.

- This method you should call on the document element, document or window itself, that is the event target.
- The ***addEventListener()*** accepts as the first argument event type, that is the name of the event without the “on” prefix.
- The second attribute is the function that should be called.
- The third attribute can be either the use capture boolean flag, default set to false or an options object.

```
let myButton = document.getElementById("myButton");
let myFunc = function () {
    alert(new Date());
};
myButton.addEventListener('click', myFunc, false);
```

- Using ***addEventListener()*** method does not overwrite the **onclick** property. Instead another event handler is created.
- The ***addEventListener()*** allows you to add multiple event handlers for the same element and type of event.
- When an event occurs on an element all related event handlers are invoked in the same order in which they were registered.
- It is possible to remove event handler registered by the ***addEventListener()*** on an element by using opposite method ***removeEventListener()***. It accepts the same three arguments and removes specified event handler.

```
myButton.removeEventListener('click', myFunc, false);
```

Calling Event Handlers

An event handler is called whenever an event of specified type occurs on specified element.

Event handler is called with an Event object passed as an argument. The event object provides information about the event. For example it contains information about event type, event target and information specific for event type like pressed key.

```
<button type="button" id="myButton">Click here</button>
```

```
function eventHandler(event) {
    console.log(event.type);
    console.log(event.target);
}

document.getElementById("myButton").onclick = eventHandler;
// Output:
// "click"
// "<button id="myButton" type="button">"
```

When an event handler is registered as HTML element attribute, the event object is available as a event variable.

```
<input type="button" onclick="alert(event.type)" value="Show event">
```

Event handler context

- When a new event handler is registered for an object, there is set an object property and a new method is defined for this object. That is why inside of an event handler you can use keyword *this*, that refers to the event target.

```
<select id="colours">

    <option value="red">red</option>
    <option value="yellow">yellow</option>
    <option value="blue">blue</option>
</select>
```

```
<div id="displayColour"></div>
```

```
let colours = document.getElementById('colours');
colours.onchange = function() {
    document.getElementById('displayColour').innerHTML = '<span
style="color: ' + this.value + '">' + this.value + '</span>';
};
```

Event propagation in Detail

Calling order

- An document element may have more than one event handler registered for a given event type.
- All these events are called in a specific order.
- First it is called event handler registered by setting an object property or HTML element attribute.
- Then are called event handlers registered by *addEventListener()* method in the same order in which they were registered.

Event bubbling

- Most of events are propagated from the most specific element towards the least specific node in the DOM tree. This feature is also called event bubbling. Consider the following example:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Event Propagation</title>
  </head>
  <body>
    <div><span>Click here</span></div>
  </body>
</html>
```

- When you click the span element, the click event occurs first on the span element, then it propagates in the following order: on the div element, on the body element, on the html element on the document object and finally on the window object.

Event capturing

- There is an alternative event flow called event capturing. In this flow the least specific node receives the event first and the most specific element receives the event at the end.
- This provides an opportunity to deal with the event before it reaches its target.
- Event capturing works only with event handlers registered with *addEventListener()* method, when the third argument is set to true.
- This feature is much less used than event bubbling.

Summarizing, there are three phases of event propagation. The first phase that occurs, is the capturing phase. Second phase is calling event handlers on the target object. The last phase is bubbling phase.

Event cancellation

— If an event handler is registered by setting an HTML element attribute or an object property the return value affects the propagation of the event. If the return value is false, then the browser's default action associated with an event is not performed.

```
<a href="http://example.com" onclick="alert('Inactive link'); return false;">Link</a>
```

— For all events, the browser's default action for an event can be canceled using *preventDefault()* method of the event object.

```
<a href="http://example.com" id="myLink">Link</a>
```

```
document.getElementById('myLink').addEventListener('click',  
function(event) {  
    event.preventDefault();  
});
```

```
    alert('This link is not active now');
}, false);
```

- The event object has *defaultPrevented* property, that is set to true if the *preventDefault()* method is called.
- It is also possible to cancel an event propagation. You should use the *stopPropagation()* method on the event object to prevent propagation of the event to any other object. Only other auction handlers registered on the same object will still be called.

Some Event Types

There are many events and their number is still growing. Some of them are known from the early days of web browsers. New ones are defined by the DOM specification led by W3C, by HTML5 APIs and by the solutions for mobile devices.

Form events

- The **submit** event is fired by the form element, when the form is submitted.
- The **reset** event is fired, when the form is reseted.
- The **click** event is fired by buttons, radio buttons and checkboxes, when the user clicks them.
- The **change** event is fired when the form element changes its state. It happens when an element is selected, a box is checked or the user has entered text into text input field and took off the focus.
- The **focus** is fired when an element gains the focus.
- The **blur** is fired when an element lose the focus.

Event Performed	Event Handler	Description
focus	onfocus	When the user focuses on an element
submit	onsubmit	When the user submits the form
blur	onblur	When the focus is away from a form element
change	onchange	When the user modifies or changes the value of a form element

Load events

- The **load** event is fired on the Window object when the document and all its external resources are fully loaded. In this state, the document is ready to be manipulated.
 - ☞ Individual document elements like img also trigger this event.
- The **DOMContentLoaded** event is fired when the document is loaded and parsed and deferred scripts executed.
- External resources like images and asynchronous scripts may still be loaded, however the document is ready to be manipulated.
- The **unload** event is fired when the document is being unloaded. It gives you the opportunity to save the user's data but cannot be used to cancel the unload.
- The **beforeunload** event is similar to the unload event and is fired when the document is about to be unloaded. At this stage, the document is still visible and cancellation is still possible.
 - Depending on the browser, if the **returnValue** event property has set a string, or a handler function returns a string, this string is displayed in a dialog asking the user for confirmation.

Window events

- Window events are mentioned above the load events.
- The focus and blur events may also be used as Window events.
- The scroll event is fired when the user scrolls the browser window.
- The resize event is fired when the user resizes the browser window.

Event Performed	Event Handler	Description
load	onload	When the browser finishes the loading of the page
unload	onunload	When the visitor leaves the current webpage, the browser unloads it
resize	onresize	When the visitor resizes the window of the browser

Mouse events

- Mouse events are triggered when the user moves or clicks mouse over the HTML document.
- The event is triggered first on the most deeply nested element, where the mouse pointer is located and it bubbles up through the document.
- The event object has several useful properties.
 - The **clientX** and the **clientY** properties specify the position of the mouse.
 - The **button** property specifies which button mouse was clicked.
 - The **detail** property specifies whether that was single or double click.
- The list of mouse events is as follow:
 - The **mousedown** is fired when the user presses a mouse button.
 - The **mouseup** is fired when the user releases a mouse button.
 - The **click** event is fired when the user press and releases a mouse button.
 - The **dblclick** is fired when user double-clicks a mouse button.
 - The **mouseover** is fired when the mouse pointer is moved onto an element.
 - The **mouseout** is fired when the mouse pointer leaves an element.
 - The **mousemove** event fires when the user moves the mouse.

Event Performed	Event Handler	Description
click	onclick	When mouse click on an element
mouseover	onmouseover	When the cursor of the mouse comes over the element
mouseout	onmouseout	When the cursor of the mouse leaves an element
mousedown	onmousedown	When the mouse button is pressed over the element

mouseup	onmouseup	When the mouse button is released over the element
mousemove	onmousemove	When the mouse movement takes place.

Mousewheel events

- The **wheel** event is fired when the user rotates the mouse wheel.
- The default action of the wheel event depends on the browser. It can be scrolling or zooming.
- The wheel event object has **deltaX**, **deltaY** and **deltaZ** properties, which specify rotation in three dimensions.

Keyboard events

- Keyboard events are generated when document element or the document itself has focus and the user presses or releases key on the keyboard.
- Keyboard events bubble up from the most specific element to the document and window.
- The **keydown** event fires when a key is pressed.
- The **keyup** event fires when a key is released.
- The **keypress** event fires after the **keydown** and before the **keyup**.
- Keyboard events objects have properties that provide details of which key has been pressed or released:
 - The **key** property value is a string containing character corresponding to key that was used or standard key value, if the key doesn't have a printable representation.
 - The **keyCode** is a legacy property specifying numerical code identifying the key.
 - The **altKey**, **ctrlKey**, **shiftKey** are properties of key events objects, that describe the state of modifier keys.

Input Event

- The input event fires when the value of an <input>, <select>, or <textarea> element has been changed.
- In JavaScript, an input event is fired whenever the value of an input field on a web page changes. This could be when a user types into a text field, selects an option from a dropdown, or chooses a file to upload
- Event name is "input" and corresponding attribute is "oninput".

Forms in JS

- A form is an HTML element that contains various input elements such as text boxes, radio buttons, checkboxes, and buttons that allow users to submit data to the server or perform some other action.
- Forms are created using HTML markup, and they can be manipulated and interacted with using JavaScript.
- JavaScript can be used to perform tasks such as validating user input, dynamically updating form fields, submitting data to the server without reloading the page, and more.
- To interact with a form using JavaScript, you can use the *document.forms* object to access the form elements by their names or indices. For example, you can access a text box element within a form using the following code:

```
let myForm = document.forms[0];
let myTextBox = myForm.elements["myTextBox"];
```

- Once you have a reference to a form element, you can modify its properties, such as its value or visibility, or add event listeners to handle user interactions such as clicks or key presses.
- We can use the **getElementById ()** method that references the html form element to the JavaScript code.

```
let form = document.getElementById('address');
```

Cookies in JS

- Cookies are small strings of data that are stored directly in the browser. They are a part of the HTTP protocol
- A cookie is a small piece of data that is stored on the user's computer by the web browser.
- Cookies are often used to store user preferences or other data that needs to persist across multiple sessions or pages on a website.
- It's important to understand that HTTP is stateless. When you send two subsequent HTTP requests to the server, there is no link between them. In other words, the server cannot know if the two requests are from the same web browser. Therefore, a cookie is used to tell if the two requests came from the same web browser.
- In practice, cookies serve the following purposes:
 - Session management – cookies allow you to manage any information that the server should remember. For example, logins, shopping carts, etc.
 - Personalization – cookies allow you to store user preferences, themes, and setting specific to a user.
 - Tracking – cookies help record and analyze user behaviors in advertising.
- Cookies are saved in name-value pairs like: `username = John Doe`
- When a browser requests a web page from a server, cookies belonging to the page are added to the request. This way the server gets the necessary data to "remember" information about users.
- To manage cookies in JavaScript, we use the **document.cookie** property.
- The **document.cookie** property allows you to read and write cookies, and it returns a string that contains all the cookies currently set for the current document.

Create a Cookie with JavaScript

- With JavaScript, a **cookie can be created** like this:

```
document.cookie = "username=John Doe";
```

You can also add an expiry date (in UTC time). By default, the cookie is deleted when the browser is closed:

```
document.cookie = "username=John Doe; expires=Thu, 28 Dec 2023  
12:00:00 UTC";
```

With a path parameter, you can tell the browser what path the cookie belongs to. By default, the cookie belongs to the current page.

```
document.cookie = "username=John Doe; expires=Thu, 28 Dec 2023  
12:00:00 UTC; path="/";
```

Read a Cookie with JavaScript

— With JavaScript, **cookies can be read** like this:

```
let x = document.cookie;
```

document.cookie will return all cookies in one string much like:

```
cookie1=value; cookie2=value; cookie3=value;
```

Change a Cookie with JavaScript

With JavaScript, you can change a cookie the same way as you create it:

```
document.cookie = "username=John Smith; expires=Thu, 28 Dec 2023  
12:00:00 UTC; path="/";
```

The old cookie is overwritten.

Delete a Cookie with JavaScript

Deleting a cookie is very simple. You don't have to specify a cookie value when you delete a cookie. Just set the expires parameter to a past date:

```
document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC;  
path=/";
```

You should define the cookie path to ensure that you delete the right cookie. Some browsers will not let you delete a cookie if you don't specify the path.

The Cookie String

- The **document.cookie** property looks like a normal text string. But it is not.
- Even if you write a whole cookie string to **document.cookie**, when you read it out again, you can only see the name-value pair of it.
- If you set a new cookie, older cookies are not overwritten. The new cookie is added to **document.cookie**, so if you read **document.cookie** again you will get something like:
cookie1 = value; cookie2 = value;
- If you want to find the value of one specified cookie, you must write a JavaScript function that searches for the cookie value in the cookie string

Handling Regular Expression in JS

- A **regular expression** is an object that describes a textual pattern. The JavaScript **RegExp** class represents regular expressions, and both **String** and **RegExp** define methods that use regular expressions to perform powerful pattern-matching and search-and-replace functions on text.
- A **regular expression** is a sequence of characters that forms a search pattern. The search pattern can be used for text search and text to replace operations.
- A regular expression can be a single character or a more complicated pattern.
- Regular expressions can be used to perform all types of text search and text replacement operations.
- Regular expression is also known as **regex** or **regexp**.

- Regular expressions in JavaScript are a powerful tool for matching, searching and manipulating text. They allow you to define a pattern of characters and use it to search for specific strings, replace parts of a string with other text, or extract data from text.

Creating regular expressions

- Regular expressions in JavaScript are represented by the **RegExp** objects.
- There are two ways of defining regular expressions and creating RegExp objects:

1. Using the RegExp constructor

- We can use the RegExp constructor to create a regular expression from a string. The constructor takes two arguments: the first is the pattern string, and the second is an optional flags string

```
const pattern = 'hello';  
  
// matches "hello" case-insensitively  
  
const regex = new RegExp(pattern, 'i');
```

- In this example, the RegExp constructor is used to create a regular expression that matches the string "hello" case-insensitively.

2. Using the literal notation

- JavaScript also has a built-in syntax for creating regular expressions using literal notation. We can enclose the regular expression pattern in forward slashes (/pattern/), and optionally add flags after the closing slash. For example:

```
const pattern = /hello/i;
```

- In this example, the regular expression `/hello/i` matches the string "hello" case-insensitively.

Regular expressions patterns

- Regular expressions consists of:

- *Ordinary characters* that represent themselves. For example the pattern `/hello world/` matches the part of the text identical with this pattern. The ordinary characters are all characters except special characters.
- *Special characters*, that are not treated literally but are interpreted in a special way. For example `'.'` (dot) matches any character except a newline.

Simple patterns

- Simple patterns consist only of ordinary characters and are used to find direct match.
- For example the pattern `/xyz/` matches only strings that contain exactly phrase `'xyz'`, with the same characters in the same order. It matches the strings: `'registration of .xyz domain'`, `'hash abcxyz123'` because both strings contains substring `'xyz'`. It does not match strings `'hash abcxy'`, `'x y z'`, because they do not contain the exact substring `'xyz'`.

Complex patterns

- Complex patterns use both ordinary and special characters. Special characters are useful when pattern needs something more than direct matching.
- For example when you need to match one or more `'x'` characters, or to match a substring in which any character is in a given position. The pattern `/x.yz/` matches substrings that contain phrase `'xyz'` preceded by any character. In the string `'abcxyz123'` the pattern matches the substring `'cxyz'`.

Literal characters and Special Characters

- All alphabetic characters and digits match themselves literally in regular expressions. JavaScript regular expression syntax also supports certain nonalphabetic characters through escape sequences that begin with a backslash (`\`). For example, the sequence `\n` matches a literal newline character in a string
- A number of punctuation characters have special meanings in regular expressions. They are:

`^ $ . * + ? = ! : | \ / ( ) [ ] { }`

Character	Description
\	Escape character, changes the meaning of the following character. If the backslash precedes a special character, it removes its special meaning, and it indicates that the character should be interpreted literally. For example the pattern <code>/xyz/</code> matches concatenation of any character and substring 'xyz', however <code>\.xyz/</code> matches literally substring 'xyz'. If the backslash precedes non-special character, it adds its a special meaning and indicates that the character should be interpreted in a special way. For example <code>/w/</code> normally matches character 'w' in strings. <code>/\w/</code> matches words, more precisely matches alphanumeric characters and underscore '_'.
.	Matches any single character except newline character.
*	Matches the preceding expression 0 or more times.
+	Matches the preceding expression 1 or more times.
?	Matches the preceding expression 0 or 1 time. If it appears after quantifier <code>*</code>, <code>+</code>, <code>?</code> or <code>{}</code> it makes the quantifier non-greedy. It is also used in lookahead assertions.
^	Matches beginning of the string. When it appears as the first character in a character set, it negates this set.
\$	Matches end of the string.
	The equivalent of a logical OR.
[	Beginning of character class definition.
]	End of character class definition.
(	Beginning of subexpression.

)	End of subexpression.
{	Beginning of quantifier min/max.
}	End of quantifier min/max.

Character classes

- Character class you can use to match a character from specific set of characters. Character class you define placing one or more characters in square brackets. If the quantifier is not given after it, exactly one of the characters defined in the character class will be matched.

Character	Description
[...]	Matches one character from the character set For example: [abc] Matches one of 'a', 'b' or 'c'
[a-z]	Matches any small letter
[a-zA-Z]	Matches any letter
[0-9]	Matches any digit

[^...]	Matches one character outside the character set For example: [^0-9] Matches any character which is not a digit
---------------	---

Predefined Character classes

There is a number of predefined character classes, which are convenient shorthands for commonly used regular expressions.

Character	Description
\d	Matches any digit
\D	Matches any non-digit character
\w	Matches any word character (letter and digit)
\W	Matches any single non-word character
\s	Matches any whitespace character
\S	Matches any non-whitespace character

Regular expressions quantifiers

*	Matches the preceding expression 0 or more times, as many items as possible. For example /k*x/ matches in string 'mx mkx mmkkxx' substrings 'x', 'kx', 'kkx'. /#.*#/ matches in string "ab #cd# ef #gh#" substrings "#cd# ef #gh#"
+	Matches the preceding expression 1 or more times, as many items as possible. For example /p+x/ matches in string 'mx mpx mmpxx' substrings 'px', 'ppx'. /#.+#/ matches in string "ab #cd# ef #gh#" substrings "#cd# ef #gh#"
?	Matches the preceding expression 0 or 1 time. For example /c?x/ matches in the string 'ccx yxx' substrings 'cx', 'x'. /abc?/ matches abc or ab.
{m}	Matches the preceding expression exactly m times
{m,}	Matches the preceding expression at least m times
{m,n}	Matches the preceding expression between m and n times
{,n}	Matches the preceding expression between 0 and n times
[...]	Matches one of the characters from the set
[^...]	Matches one outside the characters from the set
A B	Matches A or B

Non-greedy quantifiers

Non-greedy (lazy) quantifiers match as little characters as possible.

?	Matches the preceding expression 0 or more times, lazy version. For example /#.?#/ matches in string "ab #cd# ef #gh#" substrings "#cd#" and "#gh#".
+?	Matches the preceding expression 1 or more times, lazy version. For example /#.+?#/ matches in string "ab #cd# ef #gh#" substrings "#cd#" and "#gh#".
??	Matches the preceding expression 0 or 1 time, lazy version. For example /abc??/ matches ab or abc.
{m,n}?	Matches the preceding expression between m and n times, lazy version.

Anchors

^	Matches beginning of the string. For example <code>/^alpha/</code> matches substring 'alpha' in string 'alpha beta' but does not match it in the string 'the alpha'.
\$	Matches end of the string. For example <code>/alpha\$/</code> matches substring 'alpha' in the string 'the alpha' but does not match it in the string 'alpha beta'.

Regular expressions flags

g	Global match the pattern repeatedly in the string,
i	Case insensitive matching,
m	Multiple lines search,
u	Unicode, treats characters as unicode points,
y	Sticky search, starting at the current position.

Using regular expressions in JavaScript

- Regular expressions in JavaScript are used with two object types: **RegExp** and **String**.

RegExp methods

Method	Description
exec()	Executes a search for a match in a string. It returns an array of information or null on a mismatch.
test()	Tests for a match in a string. It returns true or false.

- The **exec()** method executes a regular expression to match it in a given string. It takes string as the argument and searches it to match the pattern. If it find matching, it returns

an array. The first element of the array contains the string that matched the pattern. Possible subsequent elements contain the substrings that matched subexpressions in parenthesis

Example:

```
const myRe = new RegExp('(a.c)d');
const myArray = myRe.exec('eaxcda');
console.log(myArray); // [ "axcd", "axc" ]
```

- The **exec()** method always applies a single match and returns information about that match. This requires using the global g flag and calling the exec method repeatedly to find all matches. When the **exec()** method is executed on a regular expression that has the g flag, it sets the lastIndex property of the regexp object to the position immediately following matched substring. When the **exec()** is executed next time on the same regexp, it starts search at the position specified by the lastIndex property. If it does not find the match, it set the lastIndex to 0.

```
let myRe = new RegExp('(a.c)d', 'g');
let myArray = myRe.exec('eaxcdacdaazcdc');
console.log(myArray); // [ "axcd", "axc" ]
console.log(myRe.lastIndex); // 5
```

```
myArray = myRe.exec('eaxcdacdaazcdc');
console.log(myArray); // [ "aycd", "ayc" ]
console.log(myRe.lastIndex); // 9
```

```
myArray = myRe.exec('eaxcdacdaazcdc');
console.log(myArray); // [ "azcd", "azc" ]
console.log(myRe.lastIndex); // 17
```

```
myArray = myRe.exec('eaxcdacdaazcdc');
console.log(myArray); // null
console.log(myRe.lastIndex); // 0
```

- The **test()** method takes a string as its argument and returns true if the regular expression was matched in the string and false otherwise.

```
const myRe = /a.cd/;
const result = myRe.test('eaxcda');
```

```
console.log(result); // true
```

String methods

Method	Description
match()	Returns an array containing all of the matches, including capturing groups, or null if no match is found.
matchAll()	Returns an iterator containing all of the matches, including capturing groups.
search()	Tests for a match in a string. It returns the index of the match, or -1 if the search fails.
replace()	Executes a search for a match in a string, and replaces the matched substring with a replacement substring.
replaceAll()	Executes a search for all matches in a string, and replaces the matched substrings with a replacement substring.
split()	Uses a regular expression or a fixed string to break a string into an array of substrings.

- The **search()** method takes a regular expression as its argument and returns the starting position of the first matching substring. If there is no match it returns -1

```
const text = 'Have passions for JavaScript and pass the exam';
const position = text.search(/pass/);
console.log(position); // 5
```

In the same way it works with the string argument

```
const position = text.search('pass');
console.log(position); // 5
```

- The **match()** method takes a regular expression argument and returns an array containing the result of the match. The first element of the array is the first matching string. Subsequent elements are substrings matching subexpressions in parentheses.

```
let text = 'eaxcdacdaazcdc';

let myArray = text.match(/(a.c)d/);

console.log(myArray); // [ "axcd", "axc" ]
```

If the regular expression has the global g flag set, the **match()** returns an array with all matches.

```
let text = 'eaxcdacycdacdaazcdc';
let myArray = text.match(/(a.c)d/g);
console.log(myArray); // [ "axcd", "aycd", "azcd" ]
```

- The **replace()** method takes a regular expression as the first argument and a replacement string as the second argument. It searches the string on which it is called and replaces matched substring by the given replacement string.

```
const text = 'The car is mine and the bike is yours.';
const result = text.replace(/is/, 'was');
console.log(result); // The car was mine and the bike is yours.
```

By default the **replace()** replaces only the first match. If the regular expression has the global g flag set it replaces all matches in the string by the replacement string.

```
const text = 'The car is mine and the bike is yours.';
const result = text.replace(/is/g, 'was');
console.log(result); // The car was mine and the bike was yours
```

The **replace()** allows you to refer to the matching subexpressions in parentheses in the replacement string. Substrings that match subexpressions are numbered and remembered. You can refer to them using the \$ character followed by a number that indicates specific match. You can use this feature to swap substrings matching a pattern.

```
let text = 'we are student of bca and we like bca';
let myPattern = /bca/;
let result = text.replace(myPattern, 'TU');
console.log(result); // we are student of TU and we like bca
```

- The **split()** method breaks the string on which it is called into substrings using the passed argument as the separator. It returns an array of resultant substrings.

```
let s = "123-45-678-10";

let myPattern = /-/;

let res = s.split(myPattern);
```

```
console.log(res);// ['123', '45', '678', '10']
```

Client Side Validation

- Client-side validation refers to the process of validating user input data on the client-side, typically using JavaScript, before sending it to the server for further processing. The goal of client-side validation is to provide quick feedback to the user regarding input errors, reducing the number of server-side requests and improving the overall user experience.
- Client-side validation is an initial check and an important feature of good user experience; by catching invalid data on the client-side, the user can fix it straight away. If it gets to the server and is then rejected, a noticeable delay is caused by a round trip to the server and then back to the client-side to tell the user to fix their data.

❖ NOTE: However, client-side validation should not be considered an exhaustive security measure! Your apps should always perform security checks on any form-submitted data on the server-side as well as the client-side, because client-side validation is too easy to bypass, so malicious users can still easily send bad data through to your server.

*Before submitting data to the server, it is important to ensure all required form controls are filled out, in the correct format. This is called **client-side form validation**, and helps ensure data submitted matches the requirements set forth in the various form controls*

- Some common examples of client-side validation include:

- ☞ **Form validation:** Ensuring that all required fields are filled, validating email addresses, validating password complexity, and more.
- ☞ **Input validation:** Ensuring that the user's input matches a certain format or meets certain criteria, such as only allowing numbers in a certain field.
- ☞ **File validation:** Validating the file type and size before uploading it to the server.

Types of client side validation

There are two different types of client-side validation that you'll encounter on the web:

1. **Built-in form validation** uses HTML form validation features, which we've discussed in many places throughout this module. This validation generally doesn't require much JavaScript. Built-in form validation has better performance than JavaScript, but it is not as customizable as JavaScript validation.
2. **JavaScript validation** is coded using JavaScript. This validation is completely customizable, but you need to create it all (or use a library).

Using built-in form validation

- ❖ One of the most significant features of *modern form controls* is the ability to validate most user data without relying on JavaScript. This is done by using validation attributes on form elements. Some of them are:
 - ☑ **required:** Specifies whether a form field needs to be filled in before the form can be submitted.
 - ☑ **minlength** and **maxlength:** Specifies the minimum and maximum length of textual data (strings).
 - ☑ **min** and **max:** Specifies the minimum and maximum values of numerical input types.

- ☑ **type**: Specifies whether the data needs to be a number, an email address, or some other specific preset type.
- ☑ **pattern**: Specifies a regular expression that defines a pattern the entered data needs to follow.

If the data entered in a form field follows all of the rules specified by the above attributes, it is considered valid. If not, it is considered invalid.

- ❖ When an element is valid, the following things are true:
 - ☑ The element matches the **:valid** CSS *pseudo-class*, which lets you apply a specific style to valid elements.
 - ☑ If the user tries to send the data, the browser will submit the form, provided there is nothing else stopping it from doing so (e.g., JavaScript).
- ❖ When an element is invalid, the following things are true:
 - ☑ The element matches the **:invalid** CSS *pseudo-class*, and sometimes other UI pseudo-classes (e.g., **:out-of-range**) depending on the error, which lets you apply a specific style to invalid elements.
 - ☑ If the user tries to send the data, the browser will block the form and display an error message.

Constraint Validation HTML Input Attributes

Attribute	Description
<i>disabled</i>	Specifies that the input element should be disabled
<i>max</i>	Specifies the maximum value of an input element
<i>min</i>	Specifies the minimum value of an input element
<i>pattern</i>	Specifies the value pattern of an input element
<i>required</i>	Specifies that the input field requires an element
<i>type</i>	Specifies the type of an input element

Constraint Validation CSS Pseudo Selectors

Selector	Description
<i>:disabled</i>	Selects input elements with the "disabled" attribute specified
<i>:invalid</i>	Selects input elements with invalid values

:optional	Selects input elements with no "required" attribute specified
:required	Selects input elements with the "required" attribute specified
:valid	Selects input elements with valid values

```

<html>
<head>
  <title>FormValidation</title>
  <style>
    input:invalid {
      border: 1px solid red;
    }

    input:valid {
      border: 2px solid green;
    }
  </style>
</head>
<body>
  <form>
    <label> Name:  </label>
    <input type="text" maxlength="12" required>
    <br><br>
    <label> Age:  </label>
    <input type="number" min="1" max="125">
    <br><br>
    <label> Gender:  </label>
    <select>
      <option> Male </option>
      <option> Female </option>
      <option> Other </option>
    </select> <br><br>
    <label> Website:  </label>
    <input type="url">
    <br><br>
    <label> Username:  </label>
    <input type="text" required maxlength="20">
    <br><br>
    <label> Password:  </label>
  </form>
</body>
</html>

```

```

        <input type="password" required>
        <br><br>
        <label> Confirm Password: </label>
        <input type="password" required>
        <br><br><br>
        <input type="submit">

    </form>
</body>
</html>

```

Validating forms using JavaScript

- JavaScript provides several validation methods and techniques that can be used to validate user input on the client-side.
- The form validation process typically consists of two parts— the required fields validation which is performed to make sure that all the mandatory fields are filled in, and the data format validation which is performed to ensure that the type and format of the data entered in the form is valid.
- We can use **Constraint validation API** for validation or the validation can be done without using this API.

Constraint Validation DOM Methods

Method	Description
<i>checkValidity()</i>	Returns true if an input element contains valid data.
<i>setCustomValidity()</i>	Sets the validationMessage property of an input element.

Constraint Validation DOM Properties

Property	Description
<i>validity</i>	Contains boolean properties related to the validity of an input element.
<i>validationMessage</i>	Contains the message a browser will display when the validity is false.
<i>willValidate</i>	Indicates if an input element will be validated.

```
<!DOCTYPE html>
<html>
  <body>
    <h2>JavaScript Validation</h2>
    <p>Enter a number and click OK:</p>
    <input id="id1" type="number" min="100" max="300" required />
    <button onclick="myFunction()">OK</button>
    <p>
      If the number is less than 100 or greater than 300, an error message will
      be displayed.
    </p>
    <p id="demo"></p>
    <script>
      function myFunction() {
        const inpObj = document.getElementById("id1");
        if (!inpObj.checkValidity()) {
          document.getElementById("demo").innerHTML = inpObj.validationMessage;
        } else {
          document.getElementById("demo").innerHTML = "Input OK";
        }
      }
    </script>
  </body>
</html>
```

Validity Properties

- The validity property of an input element contains a number of properties related to the validity of data:

Property	Description
<i>customError</i>	Set to true, if a custom validity message is set.
<i>patternMismatch</i>	Set to true, if an element's value does not match its pattern attribute.
<i>rangeOverflow</i>	Set to true, if an element's value is greater than its max attribute.
<i>rangeUnderflow</i>	Set to true, if an element's value is less than its min attribute.
<i>stepMismatch</i>	Set to true, if an element's value is invalid per its step attribute.
<i>tooLong</i>	Set to true, if an element's value exceeds its maxLength attribute.
<i>typeMismatch</i>	Set to true, if an element's value is invalid per its type attribute.
<i>valueMissing</i>	Set to true, if an element (with a required attribute) has no value.
<i>valid</i>	Set to true, if an element's value is valid.

```

<!DOCTYPE html>
<html>

<body>

    <h2>JavaScript Validation</h2>
    <p>Enter a number and click OK:</p>
    <input id="id1" type="number" max="100">
    <button onclick="myFunction()">OK</button>
    <p>If the number is greater than 100 (the input's max attribute), an error
message will be displayed.</p>
    <p id="demo"></p>
    <script>
        function myFunction() {
            let text;
            if (document.getElementById("id1").validity.rangeOverflow) {
                text = "Value too large";
            } else {
                text = "Input OK";
            }
            document.getElementById("demo").innerHTML = text;
        }
    </script>

</body>

</html>

```

```
<!DOCTYPE html>
<html>

<body>

    <h2>JavaScript Validation</h2>

    <p>Enter a number and click OK:</p>

    <input id="id1" type="number" max="100">
    <button onclick="myFunction()">OK</button>

    <p>If the number is greater than 100 (the input's max attribute), an error
message will be displayed.</p>

    <p id="demo"></p>

    <script>
        function myFunction() {
            let text;
            if (document.getElementById("id1").validity.rangeOverflow) {
                text = "Value too large";
            } else {
                text = "Input OK";
            }
            document.getElementById("demo").innerHTML = text;
        }
    </script>

</body>

</html>
```

- In some cases, you won't be able to or won't want to use the Constraint Validation API. You're still able to use JavaScript to validate your form, but you'll just have to write your own.

To validate a form, ask yourself a few questions:

What kind of validation should I perform?

- You need to determine how to validate your data: string operations, type conversion, regular expressions, and so on. It's up to you.

What should I do if the form doesn't validate?

- This is clearly a UI matter. You have to decide how the form will behave. Does the form send the data anyway? Should you highlight the fields that are in error? Should you display error messages?

How can I help the user to correct invalid data?

- In order to reduce the user's frustration, it's very important to provide as much helpful information as possible in order to guide them in correcting their inputs. You should offer up-front suggestions so they know what's expected, as well as clear error messages.

To perform form validation in JavaScript, you can use the following steps:

- Attach an event listener to the form submit button or form element to capture when the user fills/submits the form.
- In the event listener, use JavaScript code to access the form elements (e.g. input fields) and their values.
- Validate the form elements by checking their values against certain criteria, such as required fields or expected formats. You can use regular expressions (regex) or other built-in JavaScript functions (e.g. isNaN()) to perform validation checks.
- Display error messages or other feedback to the user if validation fails.
- If validation passes, submit the form to the server for further processing.

File: myForm.html

```
<html>
<head>
  <title>MyForm</title>
</head>
<body>
  <form id="form1" onsubmit="return checkForm()">
    <label> Full Name: </label>
    <input type="text" id="fullName">
    <span style="color:red" id="span1"> </span>
    <br><br>
    <label> Age: </label>
    <input type="number" id="age">
    <span style="color:red" id="span2"> </span>
    <br><br>
    <input type="submit" value="Send">
  </form>
  <script src="myValidator.js">
  </script>
</body>
</html>
```

```

//file: myValidator.js

function validateFullName(fullName){
    const fullNamePattern = /^D*$/;
    if(fullName.length == 0){
        document.getElementById('span1').innerHTML="This is required!";
        return false;
    }
    else if (fullName.length<2 || fullName.length>50){
        document.getElementById('span1').innerHTML="Full Name length is 1 to 50";
        return false;
    }
    else if(!fullNamePattern.test(fullName)){
        document.getElementById('span1').innerHTML="Full Name cannot have
digits";
        return false;
    }
    else
        return true;
}

function validateAge(age){
    if(age.length == 0){
        document.getElementById('span2').innerHTML="This is required!";
        return false;
    }
    else if(age < 0 || age >150){
        document.getElementById('span2').innerHTML="Age must be between 0 to
150";
        return false;
    }
    else
        return true;
}

function checkForm(){
    const fullName = document.getElementById('fullName').value;
    const age = document.getElementById('age').value;
    const fullNameValid= validateFullName(fullName);
    const ageValid = validateAge(parseInt(age));
    if(fullNameValid && ageValid){
        return true;
    }
    else{
        return false;
    }
}

```

← → ↻ ⓘ http://127.0.0.1:5500/myForm.html?

Full Name: This is required!

Age: This is required!

← → ↻ ⓘ http://127.0.0.1:5500/myForm.html?

Full Name:

Age: This is required!

← → ↻ ⓘ http://127.0.0.1:5500/myForm.html?

Full Name: Full Name cannot have digits

Age: Age must be between 0 to 150